

МИНОБРНАУКИ РОССИИ
ФГБОУ ВО «ИжГТУ имени М.Т. Калашникова»
Факультет «Информационные технологии»
Кафедра «Программное обеспечение»
Направление 09.03.04 «Программная инженерия»

УДК 004.9(04)

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к выпускной квалификационной работе бакалавра на тему:
Разработка системы для организации соревнований по армрестлингу

Дипломник
студент гр. Б22-191-1

Е.В. Егоров

Руководитель
Доцент

И.О. Архипов

Консультант
ст. преподаватель

К. С. Чернышев

Нормоконтролер
Старший преподаватель

В.П. Соболева

Зав. кафедрой ПО
д. э. н., доцент

М.В. Леонов

РЕФЕРАТ

Объем пояснительной записки выпускной квалификационной работы на тему «Разработка мониторинга программных сервисов» – 117 страниц, в том числе 35 рисунков, 12 использованных источника.

Ключевые слова: армрестлинг, соревнование, система, веб-приложение, база данных, турнирная сетка, заявка.

Целью выпускной квалификационной работы является разработка веб-платформы, предназначенной для оптимизации организации и проведения соревнований по армрестлингу, упрощения процесса регистрации участников, повышения уровня взаимодействия между участниками спортивного процесса и обеспечения точного учета результатов. Актуальность темы обусловлена отсутствием единой системы для организации соревнований и невозможностью следить за турнирной сеткой.

Проектными задачами выпускной квалификационной работы являются:

- 1) реализация реляционной БД для всей системы;
- 2) реализация клиентской и серверной части;
- 3) интеграция с Gmail для отправки уведомлений на почту участников;
- 4) реализация алгоритма ведения турнирной сетки с системой выбывания после двух поражений.

При разработке использовались следующие технологии: Python 3.11, FastAPI, SQLAlchemy, Pydantic, Uvicorn, REST API, SMTP, PostgreSQL, Docker, React.ts.

В результате выполненной работы была разработана полнофункциональное веб-приложение, позволяющее участникам подавать заявки на участие в соревновании, организаторам создавать и вести турнирные сетки и сохранять результаты соревнований в системе, зрителям смотреть турнирную сетку, которая обновляется в режиме реального времени.

Новизна исследования заключается в интеграции различных технологий управления соревнованиями в единую платформу, что позволяет значительно

повысить качество организации мероприятий и сделать их более доступными для всех участников.

Разработанное решение может быть применено в любой федерации армрестлинга России для организации соревнований различного уровня.

СОДЕРЖАНИЕ

РЕФЕРАТ.....	2
ВВЕДЕНИЕ	7
1. ТРЕБОВАНИЯ К СИСТЕМЕ ДЛЯ ОРГАНИЗАЦИИ СОРЕВНОВАНИЙ ПО АРМРЕСТЛИНГУ	10
1.1. Общее описание продукта.....	10
1.1.1. Назначение объекта автоматизации	10
1.1.2. Аналитический обзор	12
1.1.3. Классы и характеристики пользователей	15
1.1.4. Ограничения дизайна и реализации	16
1.2. Функциональные требования.....	16
1.2.1. Регистрация и аутентификация пользователей	16
1.2.2. Управление ролями и правами доступа	16
1.2.3. Создание и управление соревнованиями	17
1.2.4. Регистрация на соревнования	17
1.2.5. Просмотр хода соревнований и результатов	18
1.3. Требования к данным	18
1.4. Требования к внешним интерфейсам	21
1.4.1. Пользовательский интерфейс	21
1.5. Нефункциональные требования.....	26
1.5.1. Доступность	26
1.5.2. Надежность	26
1.5.3. Требования к безопасности.....	26
1.5.4. Конфиденциальность	27
1.5.5. Масштабируемость и модульность	27
1.5.6. Требования к времени хранения данных	27

1.5.7. Требования к производительности	27
2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ ДЛЯ ОРГАНИЗАЦИИ СОРЕВНОВАНИЙ ПО АРМРЕСТЛИНГУ	28
2.1. Модель сценариев использования предметной области	28
2.2. Логическая модель данных	29
2.3. Структурное моделирование системы.....	35
2.4. Модель классов предметной области	36
3. РЕАЛИЗАЦИЯ ПРЕЦЕДЕНТОВ СИСТЕМЫ ДЛЯ ОРГАНИЗАЦИИ СОРЕВНОВАНИЙ ПО АРМРЕСТЛИНГУ	42
3.1. Реализация прецедента «Авторизация и аутентификация»	42
3.1.1. Реализация алгоритма.....	43
3.1.2. Реализация классов	45
3.1.3. Описание контрольного примера.....	46
3.2. Реализация прецедента «Регистрация участника на соревнование»	47
3.2.1. Реализация алгоритма.....	48
3.2.2. Реализация классов	49
3.2.3. Описание контрольного примера.....	50
3.3. Реализация прецедента «Проверка заявки организатором соревнований»	51
3.3.1. Реализация алгоритма.....	52
3.3.2. Реализация классов	53
3.3.3. Описание контрольного примера.....	54
3.4. Реализация прецедента «Просмотр стартовых списков»	55
3.4.1. Реализация алгоритма.....	56
3.4.2. Реализация классов	57

3.4.3. Описание контрольного примера.....	57
3.5. Реализация прецедента «Создание турнирной сетки»	57
3.5.1. Реализация алгоритма.....	58
3.5.2. Реализация классов	60
3.5.3. Описание контрольного примера.....	61
3.6. Реализация прецедента «Ведение турнирной сетки»	61
3.6.1. Реализация алгоритма.....	62
3.6.2. Реализация классов	64
3.6.3. Описание контрольного примера.....	65
3.7. Реализация прецедента «Составление итогового протокола соревнования»	65
3.7.1. Реализация алгоритма.....	66
3.7.2. Реализация классов	67
3.7.3. Описание контрольного примера.....	67
3.8. Реализация прецедента «Редактирование профиля»	68
3.8.1. Реализация алгоритма.....	68
3.8.2. Реализация классов	69
3.8.3. Описание контрольного примера.....	69
ЗАКЛЮЧЕНИЕ	71
СПИСОК ЛИТЕРАТУРЫ	73
ПРИЛОЖЕНИЕ 1	74
ТЕКСТЫ ПРОГРАММ.....	74
ПРИЛОЖЕНИЕ 2	115
РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ.....	115

ВВЕДЕНИЕ

Современное общество уделяет всё большее внимание развитию силовых видов спорта, таких как армрестлинг, рассматривая их не только как способ укрепления здоровья, но и как средство воспитания дисциплины, целеустремленности и силы духа. Армрестлинг, как динамично развивающийся вид спорта, привлекает внимание спортсменов разного возраста и уровня подготовки. Однако организация соревнований и управление спортивными мероприятиями в этой дисциплине сталкивается с рядом трудностей, связанных с ручным управлением процессами, отсутствием автоматизации и недостаточной прозрачностью взаимодействия между участниками.

Одной из ключевых проблем является сложность организации и проведения соревнований. Федерация армрестлинга Удмуртии, как и многие другие региональные организации, сталкивается с необходимостью обработки большого объема данных: регистрация участников, учет возрастных и весовых категорий, формирование турнирной сетки, ведение результатов и генерация протоколов. Традиционные методы, такие как использование таблиц Excel или ручной ввод данных, требуют значительных временных затрат и подвержены ошибкам. Кроме того, отсутствие удобного инструмента для онлайн-регистрации участников и просмотра результатов снижает уровень удовлетворённости как спортсменов, так и организаторов.

В связи с этим возникает потребность в создании специализированного веб-приложения, которое позволит автоматизировать процессы организации соревнований, обеспечит прозрачность взаимодействия между участниками, судьями и организаторами, а также повысит эффективность управления спортивными мероприятиями.

Целью данной работы является разработка веб-платформы, предназначенной для оптимизации организации и проведения соревнований по армрестлингу, упрощения процесса регистрации участников, повышения

уровня взаимодействия между участниками спортивного процесса и обеспечения точного учета результатов. Для достижения поставленной цели были определены следующие задачи:

- провести анализ текущего состояния организации соревнований по армрестлингу и выявить основные проблемы;
- разработать концепцию веб-приложения, ориентированного на взаимодействие участников, судей и организаторов;
- реализовать модуль для автоматического создания базы данных участников с учетом возрастных и весовых категорий;
- реализовать систему онлайн-регистрации и проверки статуса заявок;
- реализовать автоматическое формирование турнирной сетки и ведение хода соревнований;
- реализовать систему генерации итоговых протоколов;
- реализовать функционал онлайн-трансляции хода соревнований для зрителей.

Новизна исследования заключается в интеграции различных технологий управления соревнованиями в единую платформу, что позволяет значительно повысить качество организации мероприятий и сделать их более доступными для всех участников. Особое внимание уделяется специфике армрестлинга, включая учет возрастных и весовых категорий, спортивных званий и команд. Предлагаемое решение сочетает в себе удобство использования, автоматизацию рутинных процессов и современные технологии, такие как онлайн-регистрация, формирование сетки и трансляции.

Результаты исследования могут быть использованы Федерацией армрестлинга Удмуртии для повышения эффективности организации соревнований и улучшения качества обслуживания участников. Предлагаемое решение открывает новые возможности для цифровизации и автоматизации спортивной среды, делая её более открытой, управляемой и ориентированной на потребности пользователей. Внедрение платформы позволит:

- снизить нагрузку на организаторов за счет автоматизации рутинных процессов;
- повысить прозрачность и честность соревнований благодаря точному учету данных;
- увеличить охват аудитории за счет онлайн-функционала и трансляций;
- обеспечить удобство взаимодействия для всех участников процесса: спортсменов, судей, организаторов и зрителей.

Таким образом, предлагаемое решение имеет значительный потенциал для внедрения не только в рамках Федерации армрестлинга Удмуртии, но и в других региональных и международных организациях, занимающихся проведением соревнований по армрестлингу.

1. ТРЕБОВАНИЯ К СИСТЕМЕ ДЛЯ ОРГАНИЗАЦИИ СОРЕВНОВАНИЙ ПО АРМРЕСТЛИНГУ

1.1. Общее описание продукта

1.1.1. Назначение объекта автоматизации

Спорт является неотъемлемой частью жизни некоторых людей. Он помогает поддерживать тонус в теле, а также способствует лучшему самочувствию. Для определения сильнейших в том или ином виде спорта проводятся соревнования как в командных видах, так и в индивидуальных.

У каждого официального вида спорта существует своя федерация. В России армрестлинг представляет Федерация армрестлинга России (ФАР). Она подразделяется на регионы, и у каждого субъекта Российской Федерации есть свои представители.

В Удмуртской Республике армрестлинг представлен Региональной физкультурно-спортивной общественной организацией «Федерация армрестлинга Удмуртской Республики» (далее ФАУР). Она занимается проведением соревнований на территории Удмуртии, которые собирают множество участников разных возрастов. Чтобы участвовать на таких соревнованиях, спортсмен должен пройти сначала предварительную регистрацию: заполнить Google-форму контактными данными, указать название команды, ФИО тренера, прикрепить страховой полис. Затем секретарь соревнований вручную проверяет эти данные, вносит информацию в свою таблицу. Если информация неверна, то он связывается с участником через контактные данные и уведомляет его о некорректности заполненной заявки. На основе этих предварительных данных формируются стартовые списки, которые публикуются в группе ВКонтакте. В день соревнований спортсмен должен пройти процедуру взвешивания, а также иметь при себе паспорт (или свидетельство о рождении для участников младше 14 лет) и денежные средства на стартовый взнос. Только после этого участник официально зарегистрирован на соревнования. Весь этот процесс можно

автоматизировать и оставить на день соревнований только процедуру взвешивания и оплату стартового взноса. Также любители и болельщики данного вида спорта не могут следить за результатами, поскольку на данный момент нет возможности следить за соревнованиями онлайн, так как их ход нигде не транслируется.

Соревнования по армрестлингу чаще всего проводятся по системе с выбыванием после двух поражений. На данный момент программы для автоматизации данной турнирной системы можно найти только через их разработчиков и получить к ним доступ возможно только на платной основе.

Предлагаемое решение – веб-приложение, которое является сервисом для регистрации участников на соревнования и инструментом для проведения и трансляции хода соревнований. Также приложение будет сохранять историю прошедших турниров и иметь возможность для экспорта протоколов по их итогам.

Веб-приложение предоставляет возможность участникам соревнований зарегистрировать себя или другого человека на турнир. Секретарю соревнований доступен инструмент для просмотра заявки и уведомления участника о состоянии отправленной им формы регистрации. Для организаторов существует интерфейс для ведения турнирной сетки и конструктора соревнований, который подразумевает создание возрастных и весовых категорий. Приложение позволит зрителям следить за ходом соревнований в реальном времени. Это позволит наблюдать за победами или поражениями своего любимого спортсмена.

Единственным потребителем данной системы можно считать ФАУР. Это позволит им улучшить систему проведения соревнований, а также повысит заинтересованность данным видом спорта среди зрителей.

Таким образом, существуют следующие проблемы:

- отсутствие автоматизации регистрации участников на соревнования;
- сложный доступ к инструментам проведения соревнований;

– отсутствие трансляции хода соревнований.

Целью является автоматизирование процесса проведения соревнований по армрестлингу, транслирование хода соревнований в реальном времени, а также реализация инструмента для ведения турнирной сетки.

1.1.2. Аналитический обзор

Согласно проведенному обзору, на рынке существует значительное количество программных решений, направленных на автоматизацию спортивных мероприятий. Однако их функционал часто типичен и ограничивается стандартными возможностями, характерными для классических CRM- и ERP-систем: регистрация участников, учет заявок, создание расписаний и базовый отчет о результатах.

На данный момент многие федерации и организаторы соревнований используют подобные приложения. Однако на рынке отсутствует специализированное решение, которое бы предоставляло комплексный функционал для проведения соревнований по армрестлингу с учетом специфики этого вида спорта, включая категоризацию участников, формирование турнирной сетки, ведение результатов, генерацию протоколов, а также трансляцию хода соревнований. В то же время, интерес к армрестлингу как к дисциплине стремительно растет, что создает потребность в современных и удобных инструментах для управления соревнованиями.

Проведён сравнительный анализ. На рынке программ для проведения соревнований по армрестлингу были подобраны несколько конкурентов.

Критерии анализа сформированы на основании существующего функционала конкурентов и удобства для конечных пользователей: организаторов соревнований, участников и зрителей. Для наглядности была составлена таблица сравнительного анализа представителей решений и выбранного функционала (табл. 1).

Таблица 1

Аналитический обзор

Характеристика	Мое решение	Armscorer	Таблицы Excel	ArmGrid	Challonge
Подача заявок на соревнования	+	-	-	-	-
Цена	бесплатно	бесплатно	бесплатно	5000 руб.	бесплатно
Ведение турнирной сетки	+	+	+	+	+
Просмотр истории соревнований	+	-	-	-	+
Генерация итоговых протоколов	+	+	-	+	+
Трансляция хода соревнований	+	-	-	-	+
Язык	русский	русский	русский	английский	русский
Тип приложения	веб-приложение	desktop-приложение	Excel-файл	веб-приложение	веб-приложение
Страна разработки	РФ	РФ	РФ	США	США

Рассмотрим каждый вариант отдельно:

1) мое решение - это современное веб-приложение, разработанное специально для армрестлинга, с акцентом на удобство и комплексный функционал. Оно предлагает онлайн-регистрацию участников, автоматическое формирование турнирной сетки с учетом категорий, генерацию протоколов и онлайн-трансляции. Бесплатное использование и поддержка русского языка делают его доступным для организаторов

соревнований. Преимущества включают возможность просматривать историю соревнований и кроссплатформенность благодаря веб-формату.

2) Armscorer — это десктопное приложение, созданное для управления соревнованиями по армрестлингу. Оно предоставляет базовые функции, такие как формирование турнирной сетки и генерация протоколов, и поддерживает русский язык. Бесплатное использование делает его легкодоступным инструментом для ФАУР. Однако отсутствие онлайн-функционала (например, регистрации или трансляций) и невозможность просматривать историю соревнований ограничивают его применимость для крупных мероприятий. Десктопный формат также усложняет доступ с мобильных устройств.

3) Excel — это универсальный инструмент, который часто используется для соревнований благодаря своей доступности и простоте. Он позволяет вручную создавать турнирные сетки и управлять данными. Однако этот подход требует значительных усилий от организаторов, высок риск ошибок, а автоматизация отсутствует. Таблицы Excel не поддерживают онлайн-функционал, что делает их непрактичными для современных мероприятий по армрестлингу с большим количеством участников.

4) ArmGrid — это коммерческое веб-приложение, ориентированное на проведение соревнований по армрестлингу. Оно предлагает формирование турнирной сетки, генерацию протоколов и другие базовые функции. Однако отсутствие онлайн-регистрации, просмотра истории соревнований и трансляций ограничивает его функциональность. Кроме того, платная модель (5000 рублей) может быть неприятна для ФАУР из-за бесплатных конкурентов. Англоязычный интерфейс также усложняет использование в проведении соревнований, так как большая часть организаторов имеют слабые знания английского языка.

5) Challonge — универсальная платформа для организации турниров, которая может быть адаптирована для армрестлинга. Она предлагает удобный интерфейс для создания турнирных сеток, просмотр истории соревнований и бесплатное использование. Поддержка русского языка делает ее доступной

для ФАУР и любителей армрестлинга. Однако отсутствие специализации под армрестлинг (например, категоризации участников) и онлайн-регистрации ограничивает ее применимость. Также платформа не предоставляет функционал для трансляций.

В результате проведенного анализа можно сделать вывод о том, что представленные программы на рынке имеют как плюсы, так и минусы.

Основываясь на результатах, планируется разработать приложение, которое будет учитывать все недостатки уже имеющихся решений. Требуется совместить функционал регистрации участников на соревнования, систему проведения соревнований, генерацию итоговых протоколов, а также онлайн-трансляцию хода соревнований.

Данное приложение позволит ФАУР улучшить автоматизацию проведения соревнований, а также привлечь больше лиц в армрестлинг.

1.1.3. Классы и характеристики пользователей

Пользователям доступны следующие роли:

- 1) организатор соревнований;
- 2) участник соревнований;
- 3) зритель;
- 4) администратор веб-приложения.

Организатор соревнований – лицо, ответственное за проведение соревнований. Им может быть президент ФАУР или его секретарь.

Участник соревнований – человек возрастом от 13 лет, планирующий участвовать на соревнованиях.

Зритель – пользователь, который использует приложение для просмотра хода соревнований.

Администратор веб-приложения – лицо, которое следит за правильной работой приложения, занимается редактированием информации на сайте и исправлением возникших ошибок. Также он назначает пользователям роль организатора соревнований.

1.1.4. Ограничения дизайна и реализации

Передача данных между клиентской и серверной частью должна быть зашифрована.

Приложение не предоставляет возможность оплаты стартовых взносов.

Серверная часть должна быть разработана с использованием фреймворка Python FastAPI.

Клиентская часть должна быть разработана с использованием фреймворка React.ts.

Для хранения и записи данных должна быть использована свободная объектно-реляционная система управления базами данных PostgreSQL.

1.2. Функциональные требования

1.2.1. Регистрация и аутентификация пользователей

- незарегистрированный пользователь может зарегистрироваться в системе, указав свои ФИО, пол, дату рождения, логин и пароль;

- привязка почты – участник соревнований при регистрации в поле для логина указывает свою электронную почту для получения уведомлений о статусе заявки;

- просмотр и редактирование личных данных – участник соревнований может просматривать и редактировать информацию о себе.

1.2.2. Управление ролями и правами доступа

- назначение организаторов соревнований – администратор веб-приложения имеет возможность назначить другому аккаунту роль организатора соревнований;

- разграничение ролей: незарегистрированный пользователь (зритель), участник соревнований, организатор соревнований, администратор.

1.2.3. Создание и управление соревнованиями

- создание соревнования – организатор соревнований может создать соревнование, выбрав весовые категории, возрастные категории, пол участников, название соревнований и место проведения;
- генерация турнирной сетки – организатор соревнований может автоматически сгенерировать турнирную сетку для каждой категории на основе зарегистрированных участников;
- ввод результатов – организатор соревнований может вносить результаты поединков через веб-интерфейс;
- отчеты по соревнованиям – организатор соревнований может создавать итоговые протоколы с распределением мест по категориям;
- экспорт итоговых протоколов – пользователи могут скачать файл с итоговым протоколом соревнований;
- запись веса участника – организатор соревнований во время процедуры взвешивания может записать стартовый вес участника соревнований.

1.2.4. Регистрация на соревнования

- регистрация на соревнования – участник соревнований может отправить заявку на участие, указав контактные данные, название команды, возрастную и весовую категории;
- прикрепление файлов в заявке – участник соревнований может прикрепить файлы различных форматов (копия паспорта, страховой полис) в заявке;
- обработка заявки – организатор соревнований может подтвердить заявку в случае корректных данных либо отклонить её;
- уведомление о статусе заявки – участник соревнований получает уведомление о статусе заявки на почту, указанную при регистрации;
- управление заявками – организатор соревнований управляет процессом обработки заявок.

1.2.5. Просмотр хода соревнований и результатов

– просмотр хода соревнований – зритель в веб-приложении может следить за ходом соревнований, смотреть всю турнирную сетку и видеть результаты поединков;

– просмотр истории соревнований – зритель может просмотреть результаты прошедших соревнований.

1.3. Требования к данным

Требования к составу и формату данных представлены ниже (табл. 2).

Таблица 1

Словарь данных

Название сущности	Данные сущности
Пользователь	– ID – числовое поле; – логин – текстовое поле; – пароль – текстовое поле; – имя – текстовое поле; – фамилия – текстовое поле; – отчество – текстовое поле; – пол – текстовое поле; – дата рождения – дата; – роль – числовое поле.
Роль	– ID – числовое поле; – название роли – текстовое поле.
Спортивный разряд	– ID – числовое поле; – название – текстовое поле.

Продолжение табл. 2

Название сущности	Данные сущности
Соревнование	– ID – числовое поле; – название соревнования – текстовое поле; – место проведения – текстовое поле; – дата проведения – дата; – организатор – числовое поле.
Возрастная категория	– ID – числовое поле; – минимальный год рождения – числовое поле; – максимальный год рождения – числовое поле; – название категории – текстовое поле.
Весовая категория	– ID – числовое поле; – минимальный вес – числовое поле; – максимальный вес – числовое поле; – название категории – текстовое поле.
Заявка участника	– ID – числовое поле; – возрастная категория - числовое поле; – весовая категория - числовое поле; – пользователь – числовое поле; – соревнование – числовое поле; – вес участника – числовое поле; – пол – текстовое поле; – спортивное звание – числовое поле; – команда – текстовое поле. – время создания – дата; – время редактирования; – статус – текстовое поле.

Продолжение табл. 2

Название сущности	Данные сущности
Уведомления	– ID – числовое поле; – создатель уведомления – числовое поле; – заявка участника – числовое поле; – текст уведомления – текстовое поле.
Файл	– ID – числовое поле; – название файла – текстовое поле; – путь файла – текстовое поле; – заявка участника – числовое поле.
Поединок	– ID – числовое поле; – ID соревнования – числовое поле; – ID турнира – текстовое поле; – рука – текстовое поле; – ID возрастной категории – числовое поле; – ID весовой категории – числовое поле; – номер раунда – числовое поле; – сетка – текстовое поле; – номер поединка – числовое поле; – имя первого участника – текстовое поле; – имя второго участника – текстовое поле; – имя победителя – текстовое поле; – имя проигравшего – текстовое поле; – проходной матч – логическое поле; – дата создания – дата/время.

Продолжение табл. 2

Название сущности	Данные сущности
Турнир	– ID – числовое поле; – ID турнира – текстовое поле; – ID соревнования – числовое поле; – ID организатора – числовое поле; – рука – текстовое поле; – ID возрастной категории – числовое поле; – ID весовой категории – числовое поле; – данные участников – текстовое поле; – состояние (JSON) – текстовое поле; – активен – логическое поле; – дата создания – дата/время; – дата обновления – дата/время.
Результаты	– ID – числовое поле; – заявка участника – числовое поле; – место, занятое на левой руке – числовое поле; – место, занятое на правой руке – числовое поле; – соревнование – числовое поле.

1.4. Требования к внешним интерфейсам

1.4.1. Пользовательский интерфейс

Рекомендованный размер экрана: 1920×1080.

Основная страница (рис. 1), на которую в первую очередь попадают незарегистрированные пользователи. Здесь можно посмотреть список соревнований, а также войти в аккаунт или зарегистрироваться в системе.

Главная страница

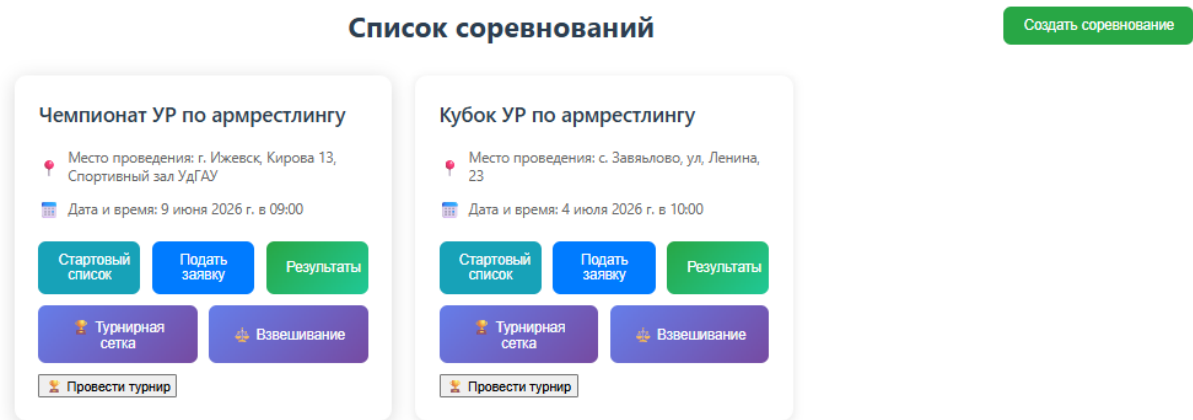


Рис. 1.1

На странице входа (рис. 2) и регистрации пользователь вводит свои данные. Система уведомляет его, если введенный им логин или пароль не соответствуют данным, указанными при регистрации. При регистрации есть проверка длины пароля и даты рождения.

Страница входа

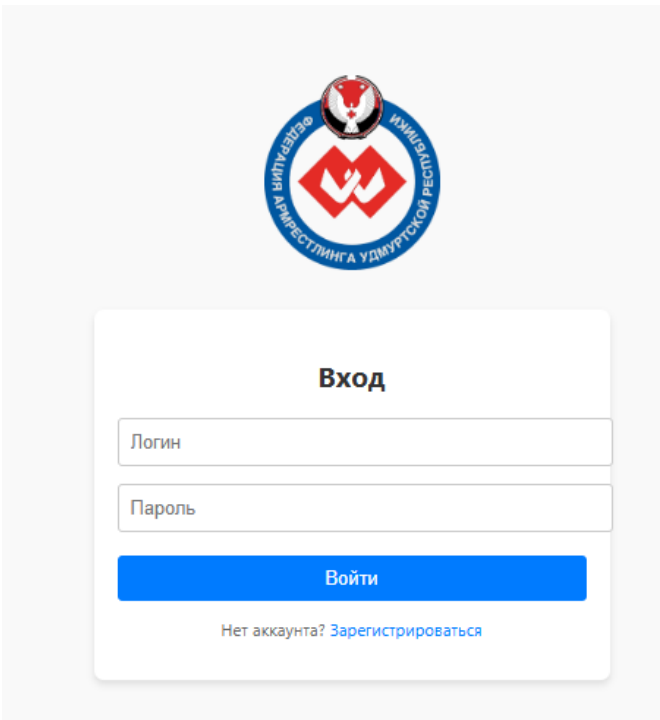


Рис. 1.2

Страница с соревнованием (рис. 3) содержит подробную информацию о нем, а именно его название и дата проведения. Также на этой странице можно просмотреть полный список участников, прошедших регистрацию на соревнования. Для каждого стола можно просмотреть турнирную сетку.

Страница соревнования

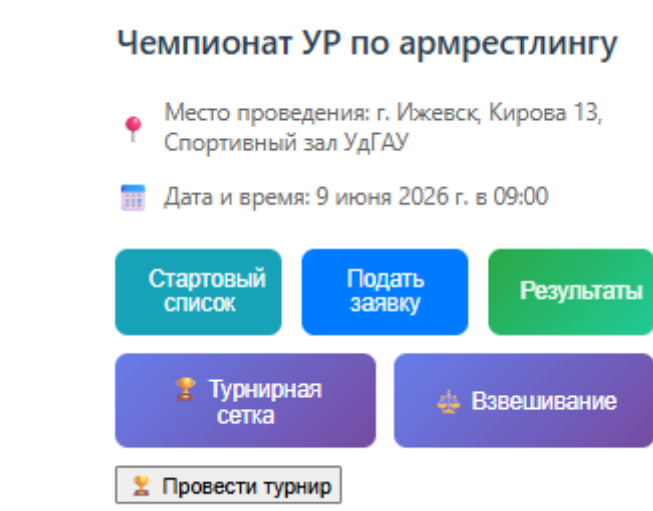


Рис. 1.3

При выборе возрастной и весовой категорий пользователь попадает на страницу (рис. 4) с полной турнирной сеткой с системой выбывания после двух поражений. На каждом этапе отображаются поединки, их участники и результаты.

Страница с турнирной сеткой

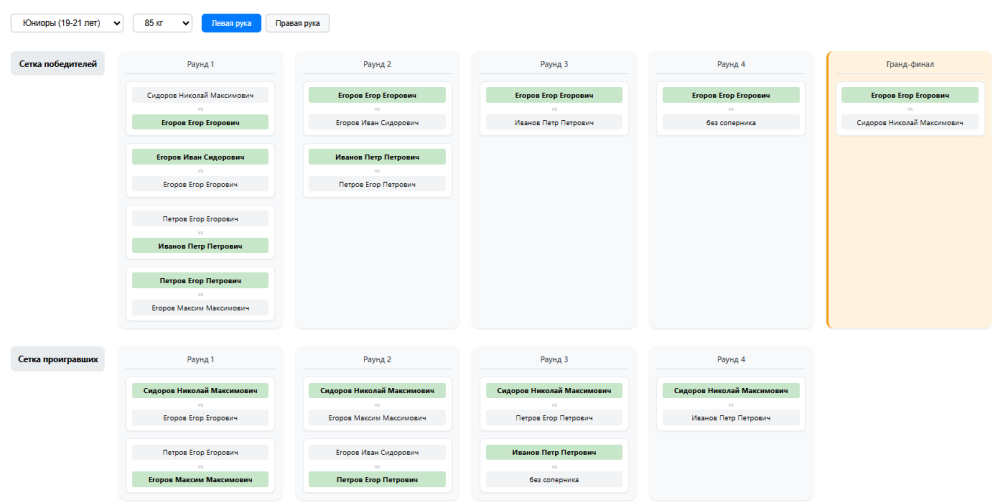


Рис. 1.4

Для регистрации на соревнования пользователь использует страницу подачи заявок (рис. 5). Участник заполняет все поля, прикрепляет необходимые файлы, а затем отправляет заявку.

Страница с формой регистрации на соревнование

Подача заявки на соревнование

Возрастная категория:

Юниоры (14-15 лет) (2011 г.р. - 2012 г.р.) ▼

Весовая категория:

45 кг (40 - 45 кг) ▼

Спортивное звание:

без разряда ▼

Команда:

Копия паспорта:

Выберите файл

Файл не выбран

Файл страхового полиса:

Выберите файл

Файл не выбран

☐ Я даю согласие на обработку моих персональных данных

Подать заявку

Рис. 1.5

Для просмотра и проверки заявок организатор использует страницу с заявками от всех участников. Здесь он может проверить корректность введенных данных, а также копию паспорта и срок действия прикрепленной страховки.

Страница с заявками

Все заявки участников

Соревнование: Чемпионат УР по армрестлингу
ФИО участника: Сидоров Николай Максимович
Возрастная категория: Юниоры (19-21 лет)
Весовая категория: 85 кг
Команда: ИЖГТУ
Создана: 05:49, 08 мая 2026
Обновлена: 05:49, 08 мая 2026
Статус: Неизвестно

Нет прикрепленных файлов.

Изменить статус: Принята ▼

Соревнование: Чемпионат УР по армрестлингу
ФИО участника: Егоров Егор Егорович
Возрастная категория: Юниоры (19-21 лет)
Весовая категория: 85 кг
Команда: ИЖГТУ
Создана: 05:49, 08 мая 2026
Обновлена: 05:49, 08 мая 2026
Статус: Неизвестно

Нет прикрепленных файлов.

Изменить статус: Принята ▼

Рис. 1.6

Для просмотра итогов соревнований есть страница (рис. 7), которая содержит список таблиц с итоговыми протоколами соревнований. В них содержится следующая информация: ФИО участника, вес участника, название команды, место на левой руке и правой, а также очки, полученные спортсменом в соответствии с местом, общее место.

Страница с результатами соревнований

← Назад

Итоговые результаты: Чемпионат УР по армрестлингу

Скачать протокол (Word)

Всего участников: 8

Юниоры (19-21 лет)

85 кг

8 участ.

МЕСТО ПО "ДВОЕБОРЬЮ"	ФАМИЛИЯ, ИМЯ, ОТЧЕСТВО	ПОЛ, ДАТА РОЖДЕНИЯ	СПОРТИВНОЕ ЗВАНИЕ	КОМАНДА	МЕСТО В БОРЬБЕ ЛЕВОЙ РУКОЙ	ОЧКИ	МЕСТО В БОРЬБЕ ПРАВОЙ РУКОЙ	ОЧКИ	СУММА ОЧКОВ	ВЕС СПОРТСМЕНА
1	Егоров Егор Егорович	муж., 04.07.2004	3 раз.	ИжГТУ	1	25		0	25	80.47
2	Иванов Петр Петрович	муж., 04.07.2004	3 юн. раз.	ИжГТУ	2	17		0	17	80.73
3	Егоров Егор Егорович	муж., 04.07.2004	3 юн. раз.	ИжГТУ	3	9		0	9	81.14
4	Егоров Иван Сидорович	муж., 04.07.2004	3 раз.	ИжГТУ	4	5		0	5	81.75
5	Егоров Максим Максимович	муж., 04.07.2004	1 юн. раз.	ИжГТУ	5	3		0	3	82.44
6	Петров Егор Егорович	муж., 04.07.2004	1 раз.	ИжГТУ	6	2		0	2	83.57
7	Петров Егор Петрович	муж., 04.07.2004	2 юн. раз.	ИжГТУ	7	0		0	0	84.22
8	Сидоров Николай Максимович	муж., 04.07.2004	3 юн. раз.	ИжГТУ	8	0		0	0	84.83

Рис. 1.7

1.5. Нефункциональные требования

1.5.1. Доступность

Веб-приложение должно быть доступно в течение 100% времени на момент проведения соревнований, и 95% времени в другие события.

1.5.2. Надежность

Система должна иметь время работы без сбоев в 99,99% с минимальным риском их появления.

1.5.3. Требования к безопасности

Заявки на участие могут отправлять только авторизованные пользователи. Создавать соревнования, редактировать информацию о них, просматривать заявки, менять их статус могут только пользователи с ролью организатора. Данный функционал выдается администратором.

Связь между веб-клиентом и серверной частью осуществляется через защищенный протокол HTTPS, который обеспечивает шифрование данных между частями веб-приложения.

1.5.4. Конфиденциальность

Пароли пользователей должны храниться в базе данных в захешированном виде для минимизации рисков и их утечки для неправомерного пользования злоумышленниками.

1.5.5. Масштабируемость и модульность

Архитектура веб-приложения должна позволять увеличивать производительность без длительной остановки работы и значительной модификации кода.

1.5.6. Требования к времени хранения данных

Информация о прошедших соревнованиях должна храниться в базе данных в течение пяти лет для просмотра истории о занятых местах тем или иным спортсменом для анализа его результатов.

1.5.7. Требования к производительности

Система должна поддерживать одновременную работу 500 клиентов.

2. ПРОЕКТИРОВАНИЕ СИСТЕМЫ ДЛЯ ОРГАНИЗАЦИИ СОРЕВНОВАНИЙ ПО АРМРЕСТЛИНГУ

2.1. Модель сценариев использования предметной области

Акторами в системе являются все пользователи, имеющую какую-либо роль в системе и доступ к прецедентам.

Существует несколько акторов:

- пользователь с ролью «зритель». Роль зрителя соревнований предоставляет доступ к такому функционалу, как просмотр турнирной сетки соревнования в реальном времени, а также просмотр истории проведения предыдущих соревнований;

- пользователь с ролью «participant». Роль участника соревнований предоставляет доступ к такому функционалу, как регистрация и авторизация в личном кабинете, подача заявок на участие в соревнованиях;

- пользователь с ролью «organizer». Роль организатора соревнований предоставляет доступ к такому функционалу, как регистрация и авторизация в личном кабинете как организатора соревнований, создание соревнования, просмотр и проверка заявок участников, ведение турнирной сетки соревнования;

- пользователь с ролью «administrator». Роль администратора предоставляет доступ к такому функционалу, как назначение организаторов соревнований и поддержка всей системы.

Диаграмма прецедентов

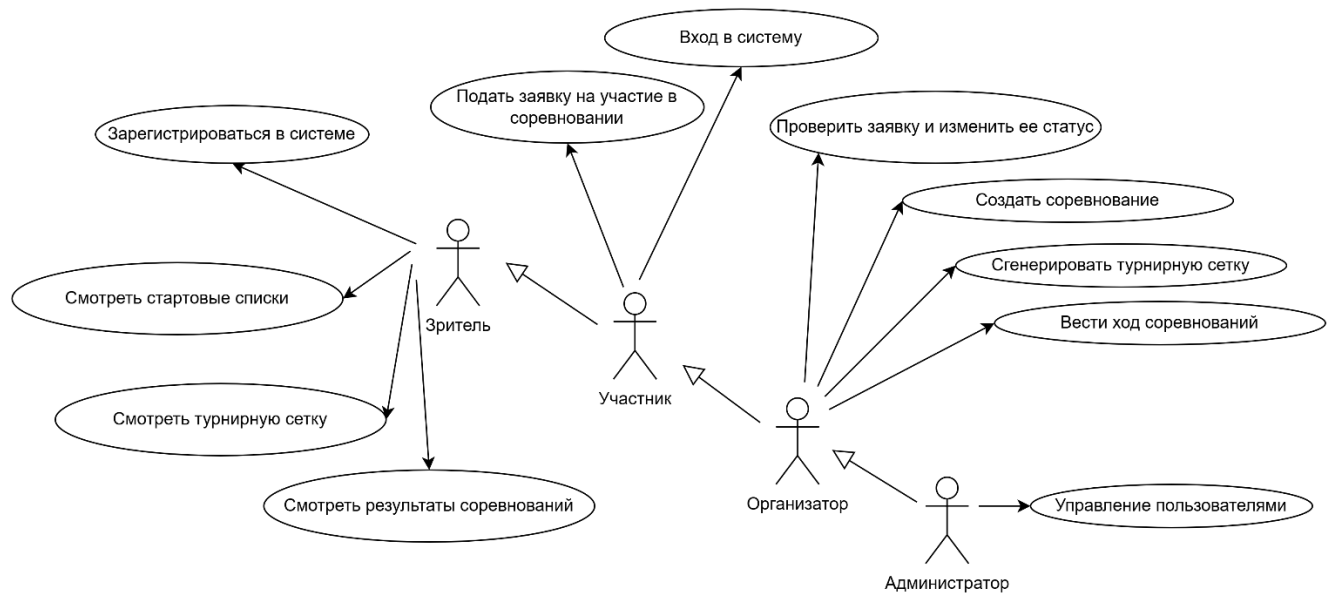


Рис. 2.1

2.2. Логическая модель данных

Логическая модель данных представляет собой 12 таблиц БД (рис. 2.2).

Таблица Users является основной таблицей в системе, так как содержит информацию обо всех пользователях системы — участниках соревнований, судьях, администраторах. Таблица Roles хранит существующие роли в системе. Таблицы Age_categories, Weight_categories и Ranks содержат информацию о возрастных, весовых категориях и спортивном разряде соответственно. Таблица Files содержит записи о прикрепленных файлах участников, такие как копия паспорта и страховой полис. Таблица Applications содержит информацию о заявках участников на соревнования. Таблица Notifications хранит в себе записи уведомлений о заявках. Таблица Competitions содержит записи о названии, месте и времени проведения соревнований. Таблицы Matches и Results отвечают за результаты и итоги поединков на соревнованиях.

Логическая модель данных

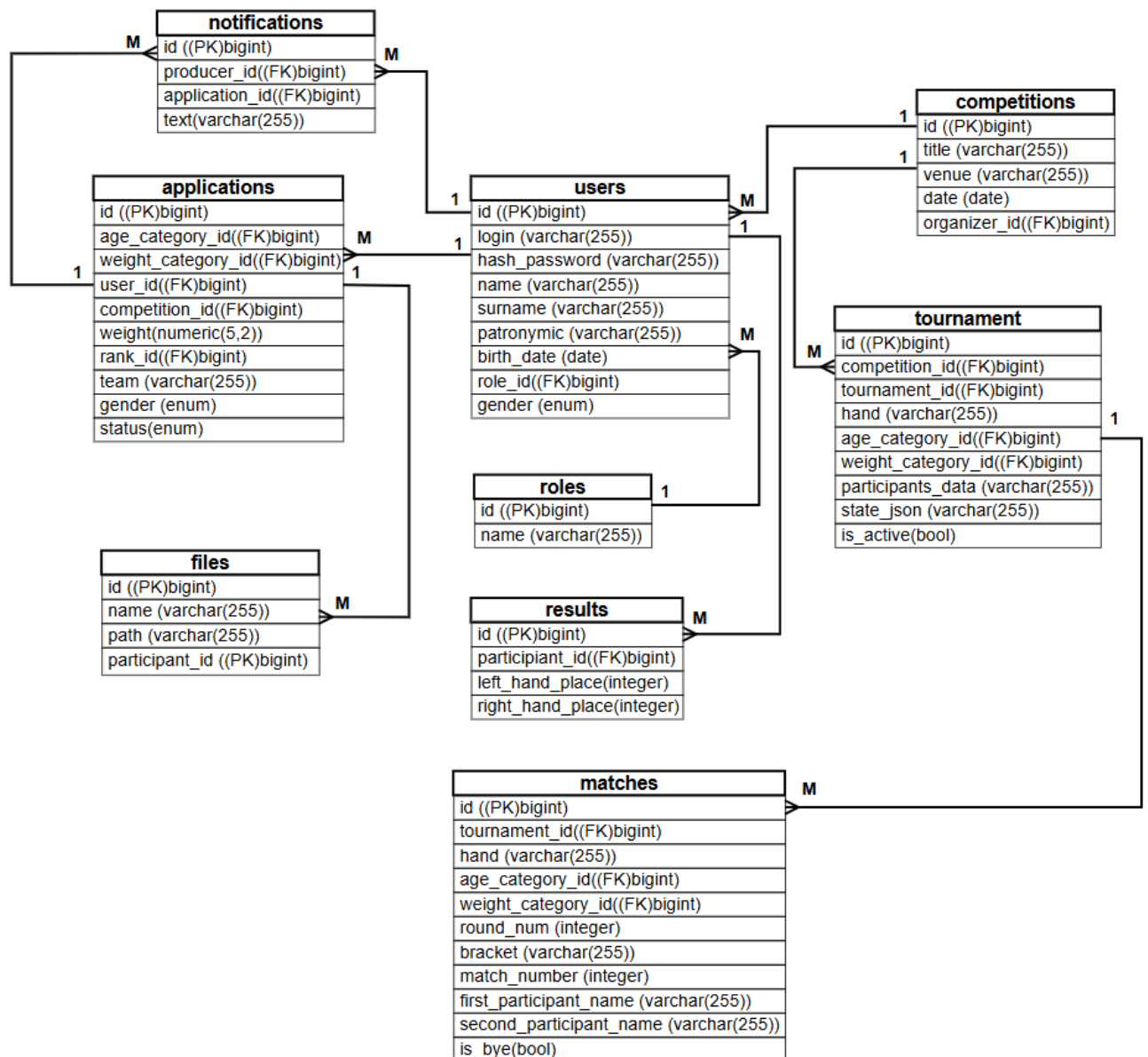


Рис 2.2

1) модель данных Пользователь (users) содержит поля:

- id (bigint, PK) - первичный ключ;
- login (varchar(255)) - логин пользователя для аутентификации;
- hash_password (varchar(255)) - хэш пароля;
- name (varchar(255)) - имя;
- surname (varchar(255)) - фамилия;
- patronymic (varchar(255)) - отчество;

- birth_date (date) - дата рождения (для определения возрастной категории);

- gender (enum) - пол;

- role_id (bigint, FK) - внешний ключ на roles. Связь M:1 (у роли много пользователей).

2) модель данных Роль (roles) содержит поля:

- id (bigint, PK) - первичный ключ для обеспечения уникальности каждого элемента таблицы реляционной БД;

- name (varchar(255)) - название роли со строковым типом данных (например, «участник», «судья», «администратор»).

3) модель данных Возрастная категория (age_categories) содержит поля:

- id (bigint, PK) - первичный ключ для обеспечения уникальности каждого элемента таблицы реляционной БД;

- name (varchar(255)) - название возрастной категории со строковым типом данных;

- min_year (integer) - минимальная граница возраста (или год рождения) для категории;

- max_year (integer) - максимальная граница возраста (или год рождения) для категории.

4) модель данных Заявка на участие (applications) содержит поля:

- id (bigint, PK) - первичный ключ;

- user_id (bigint, FK) → users. Связь M:1 (один пользователь - много заявок);

- competition_id (bigint, FK) → competitions. Связь M:1 (на одно соревнование - много заявок);

- age_category_id (bigint, FK) → age_categories. Связь M:1;

- weight_category_id (bigint, FK) → weight_categories. Связь M:1;

- rank_id (bigint, FK) → ranks. Связь M:1;

- weight (numeric(5,2)) - фактический вес участника;

- team (varchar(255)) - команда;
- gender (enum) - пол;
- status (enum) - статус заявки («одобрена», «отклонена», «на рассмотрении»);

- created_at (timestamp) - дата создания;
- updated_at (timestamp) - дата обновления.

5) модель данных Схватка (matches) содержит поля:

- id (bigint, PK) - первичный ключ;
- tournament_id (bigint, FK) → tournaments (см. ниже). Связь M:1 (в одном турнире много схваток);

- hand (varchar(255)) - рука (левая/правая);
- age_category_id (bigint, FK) → age_categories. Связь M:1;
- weight_category_id (bigint, FK) → weight_categories. Связь M:1;
- round_num (integer) - номер раунда;
- bracket (varchar(255)) - сетка (например, «верхняя», «нижняя»);
- match_number (integer) - номер матча в рамках раунда;
- first_participant_name (varchar(255)) - имя первого участника;
- second_participant_name (varchar(255)) - имя второго участника;
- is_bye (bool) - признак технической победы.

6) модель данных Результат участия (results) содержит поля:

- id (bigint, PK) - первичный ключ для обеспечения уникальности каждого элемента таблицы реляционной БД;

- participant_id (bigint, FK) - внешний ключ для идентификации участника соревнования. Отношение results:applications имеет связь 1:1, то есть один-к-одному, потому что участник соревнований имеет единственный результат на соревновании;

- left_hand_place (integer) - место, занятое участником на левой руке;
- right_hand_place (integer) - место, занятое участником на правой руке.

7) модель данных Файл (files) содержит поля:

- id (bigint, PK) - первичный ключ для обеспечения уникальности каждого элемента таблицы реляционной БД;

- name (varchar(255)) - оригинальное имя файла со строковым типом данных;

- path (varchar(255)) - путь к файлу на сервере или в облачном хранилище;

- participant_id (bigint, FK) - внешний ключ для идентификации участника, которому принадлежит файл (например, загруженная справка или фото). Отношение files:applications имеет связь M:1, то есть многие-к-одному, потому что участник соревнований может загрузить несколько документов.

8) модель данных Уведомление (notifications) содержит поля:

- id (bigint, PK) - первичный ключ для обеспечения уникальности каждого элемента таблицы реляционной БД;

- producer_id (bigint, FK) - внешний ключ для идентификации отправителя уведомления (система или администратор). Отношение notifications:applications имеет связь M:1, то есть многие-к-одному, потому что заявка участника может иметь несколько уведомлений;

- application_id (bigint, FK) - внешний ключ для идентификации заявки, по которой отправлено уведомление. Отношение notifications:users имеет связь M:1, то есть многие-к-одному, потому что организатор соревнований может отправить несколько уведомлений;

- text (varchar(255)) - текст уведомления со строковым типом данных.

9) модель данных Соревнование (competitions) содержит поля:

- id (bigint, PK) - первичный ключ для обеспечения уникальности каждого элемента таблицы реляционной БД;

- title (varchar(255)) - название соревнования;

- venue (varchar(255)) - адрес проведения соревнования;

- date (date) – дата проведения соревнования;

– organizer_id (bigint, FK) – внешний ключ для идентификации организатора соревнований. Отношение competitions:users имеет связь 1:1, то есть один-к-одному, потому у соревнования может быть один организатор.

10) модель данных Разряд (ranks) содержит поля:

– id (bigint, PK) - первичный ключ для обеспечения уникальности каждого элемента таблицы реляционной БД;

– name (varchar(255)) – наименование спортивного разряда.

11) модель данных Весовая категория (weight_categories) содержит поля:

– id (bigint, PK) - первичный ключ для обеспечения уникальности каждого элемента таблицы реляционной БД;

– name (varchar(255)) - название весовой категории со строковым типом данных;

– min_weight (numeric(5,2))- минимальная граница веса (с точностью до сотых килограмма) спортсмена для категории;

– max_weight (numeric(5,2))- максимальная граница веса (с точностью до сотых килограмма) спортсмена для категории.

12) модель данных Турнир (tournaments) содержит поля:

– id (bigint, PK) - первичный ключ;

– competition_id (bigint, FK) → competitions. Связь M:1 (в одном соревновании несколько турниров);

– tournament_id (bigint, FK) - внешний ключ на саму tournaments (для иерархии). Связь M:1 (один родительский турнир может включать несколько дочерних);

– hand (varchar(255)) - рука;

– age_category_id (bigint, FK) → age_categories. Связь M:1;

– weight_category_id (bigint, FK) → weight_categories. Связь M:1;

– participants_data (varchar(255)) - данные об участниках (например, JSON);

- state_json (varchar(255)) - текущее состояние турнира (сетка, прогресс);
- is_active (bool) - активен ли турнир.

2.3. Структурное моделирование системы

Структурная схема

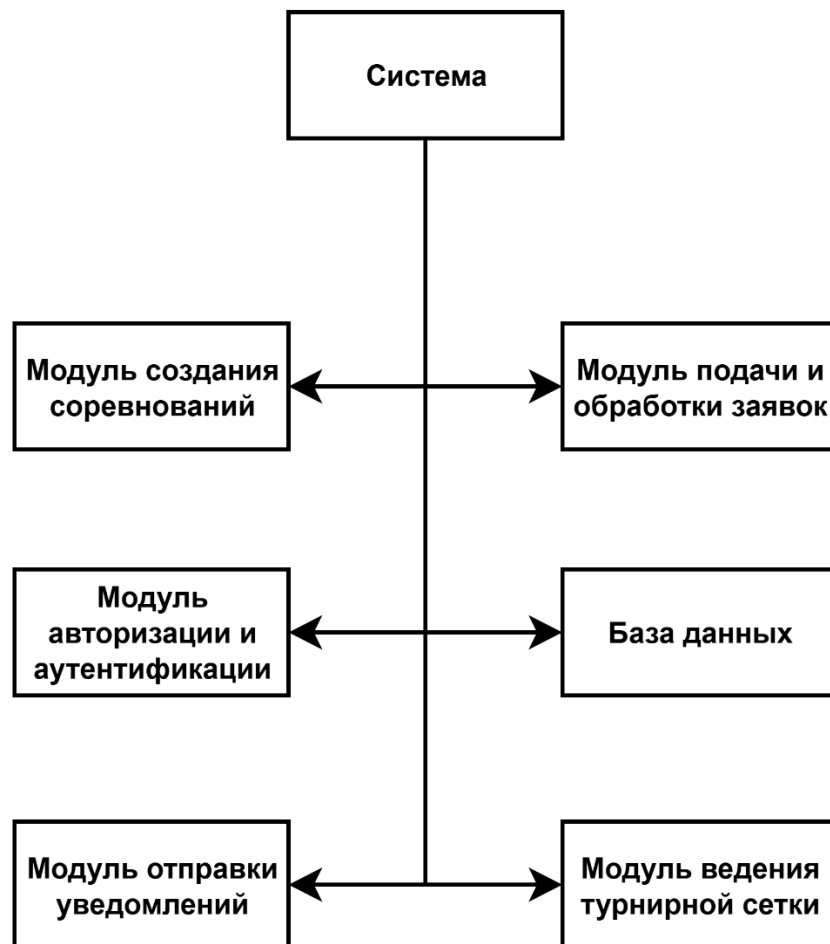


Рис. 2.3

Модель создания соревнований предназначена для пользователей с ролью «Организатор». В этом модуле реализована работа с соревнованиями, а именно их создание с указанием места и времени проведения, а также отображения списка соревнований на главной странице сайта.

Модуль авторизации и аутентификации предназначен для регистрации пользователей в системе. Управляет процессом выдачи токенов на основе JWT, а также определением роли пользователя.

Модуль подачи и обработки заявок служит участникам и организаторам для работы с процессом предварительной регистрации на соревнования. Участникам доступны подача заявки на соревнование и просмотр собственных заявок. Организатор имеет функционал просмотра всех заявок и их обработки.

Модуль отправки уведомлений служит для связи организатора с участниками. При обработке заявки участника отправляется сообщение на его почту с информацией о состоянии заявки (принята, либо отклонена).

Модуль ведения турнирной сетки предназначен для проведения соревнований. Он реализует создание турнирных сеток для каждой категории, а также ведение сетки с системой выбывания после двух поражений. Здесь же реализован просмотр текущего состояния турнирной сетки для зрителей.

База данных служит для хранения и создания записей, необходимых для работы системы.

2.4. Модель классов предметной области

Модель классов реализована в виде 8 основных классов, соответствующих таблицам базы данных (рис. 2.4). Центральными классами системы являются User (пользователь) и Competition (соревнование), которые связывают остальные сущности. Класс Application выступает в роли заявочной сущности, связывающей пользователя с соревнованием. Классы AgeCategory, WeightCategory и Rank являются справочными и определяют категории участников. Класс Tournament определяет турнирные сетки в рамках одного соревнования. Класс Match отвечает за хранение информации о поединках, а Result фиксирует итоговые места участников. Класс File обеспечивает хранение прикрепленных документов, а Notification — историю уведомлений.

Для аутентификации и разграничения доступа используются классы User и Role. Класс Role определяет права пользователя в системе (зритель, участник, организатор, администратор).

Нотификации обрабатываются классом Notification, который связан с заявками участников. Файлы, прикрепляемые к заявкам (копия паспорта, страховой полис), хранятся в классе File.

Модель классов предметной области

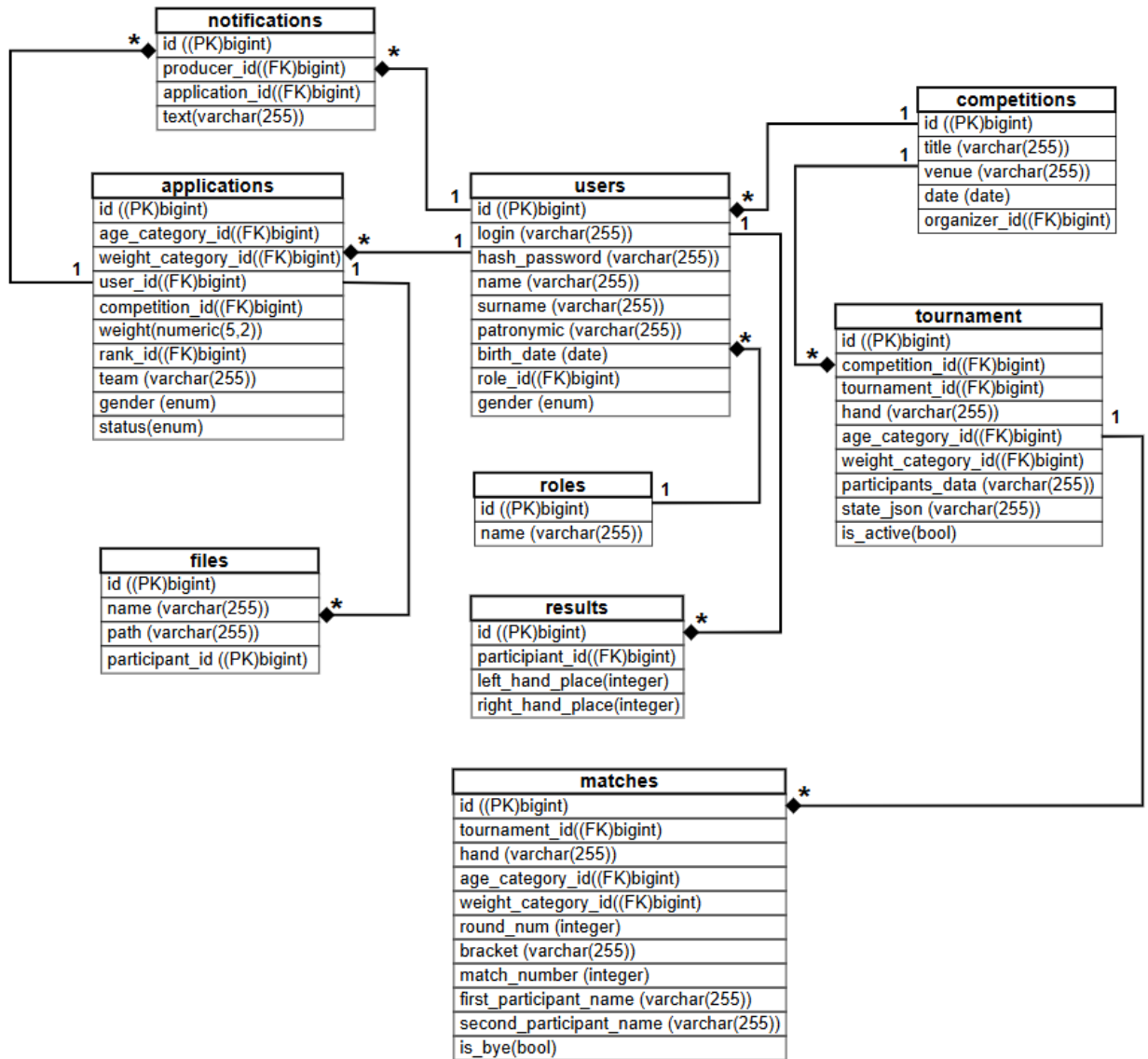


Рис. 2.4

Опишем классы:

1) пользователь (User) с атрибутами:

- id (int) - уникальный идентификатор пользователя;
- login (str) - логин для аутентификации в системе;
- hash_password (str) - хэш пароля;

- name (str) - имя пользователя;
- surname (str) - фамилия пользователя;
- patronymic (str | None) - отчество пользователя;
- birth_date (date) - дата рождения пользователя;
- gender (Enum['male', 'female']) - пол пользователя;
- role_id (int) - внешний ключ на роль пользователя.

2) роль (Role) с атрибутами:

- id (int) - уникальный идентификатор роли;
- name (str) - название роли («зритель», «участник», «организатор»,

«администратор»).

3) возрастная категория (AgeCategory) с атрибутами:

- id (int) - уникальный идентификатор возрастной категории;
- name (str) - название возрастной категории;
- min_year (int) - минимальная граница года рождения для категории;
- max_year (int) - максимальная граница года рождения для категории.

4) весовая категория (WeightCategory) с атрибутами:

- id (int) - уникальный идентификатор весовой категории;
- name (str) - название весовой категории;
- min_weight (float | None) - минимальный вес для категории;
- max_weight (float | None) - максимальный вес для категории.

5) соревнование (Competition) с атрибутами (на основе текста работы):

- id (int) - уникальный идентификатор соревнования;
- name (str) - название соревнования;
- location (str) - место проведения;
- date (date) - дата проведения;
- organizer_id (int) - внешний ключ на пользователя-организатора.

6) заявка участника (Application) с атрибутами:

- id (int) - уникальный идентификатор заявки;
- age_category_id (int) - внешний ключ на возрастную категорию;

- weight_category_id (int) - внешний ключ на весовую категорию;
- user_id (int) - внешний ключ на пользователя-участника;
- competition_id (int) - внешний ключ на соревнование;
- weight (float) - фактический вес участника на момент подачи заявки;
- rank_id (int) - внешний ключ на спортивный разряд;
- team (str | None) - название команды участника;
- gender (Enum['male', 'female']) - пол участника;
- created_at (datetime) - дата и время создания заявки;
- update_at (datetime) - дата и время последнего обновления заявки;
- status (Enum['pending', 'accepted', 'rejected']) - статус заявки.

7) спортивный разряд (Rank) с атрибутами:

- id (int) - уникальный идентификатор спортивного разряда;
- name (str) - название разряда.

8) поединок (Match) с атрибутами:

- id (int | None) - уникальный идентификатор матча;
- competition_id (int) - идентификатор соревнования, к которому

относится матч;

- tournament_id (str) - идентификатор турнира (текстовый);
- hand (str) - рука, на которой проводится поединок;
- age_category_id (int) - идентификатор возрастной категории;
- weight_category_id (int) - идентификатор весовой категории;
- round_num (int) - номер раунда турнирной сетки;
- bracket (str) - сетка, в которой находится матч;
- match_number (int) - порядковый номер матча в рамках раунда;
- first_participant_name (str) - имя первого участника;
- second_participant_name (str | None) - имя второго участника (если

есть);

- winner_name (str | None) - имя победителя матча;
- loser_name (str | None) - имя проигравшего матча;

– is_bye (bool) - флаг «технической победы» (техническая победа без соперника);

– created_at (datetime | None) - дата и время создания записи.

9) результат участия (Result) с атрибутами:

– id (int) - уникальный идентификатор результата;

– participant_id (int) - внешний ключ на участника (заявку);

– left_hand_place (int | None) - место, занятое на левой руке;

– right_hand_place (int | None) - место, занятое на правой руке;

– competition_id (int) - внешний ключ на соревнование.

10) файл (File) с атрибутами:

– id (int) - уникальный идентификатор файла;

– name (str) - оригинальное имя файла;

– path (str) - путь к файлу на сервере;

– participant_id (int) - внешний ключ на участника (заявку).

11) уведомление (Notification) с атрибутами:

– id (int) - уникальный идентификатор уведомления;

– producer_id (int) - внешний ключ на отправителя уведомления;

– application_id (int) - внешний ключ на заявку;

– text (str) - текст уведомления.

12) турнирная сетка (Tournament) с атрибутами:

– id (int) - первичный ключ матча;

– tournament_id (str) - уникальный строковый идентификатор турнира;

– competition_id (int) - внешний ключ на соревнование (competitions.id);

– organizer_id (int) - внешний ключ на пользователя-организатора (users.id);

– hand (str) - рука, для которой ведется турнирная сетка;

– age_category_id (int) - внешний ключ на возрастную категорию;

– weight_category_id (int) - внешний ключ на весовую категорию;

– participants_data (str) - данные об участниках матча;

- state_json (str | None) - состояние матча в формате JSON (сетка, счёт, прогресс);
- is_active (bool) - флаг активности матча (активен/завершён);
- created_at (datetime) - дата и время создания записи;
- updated_at (datetime) - дата и время последнего обновления записи.

3. РЕАЛИЗАЦИЯ ПРЕЦЕДЕНТОВ СИСТЕМЫ ДЛЯ ОРГАНИЗАЦИИ СОРЕВНОВАНИЙ ПО АРМРЕСТЛИНГУ

3.1. Реализация прецедента «Авторизация и аутентификация»

При регистрации пользователь указывает в форме регистрации свои данные, а именно ФИО, дату рождения, пол. Также необходимо придумать логин и пароль. Далее происходит проверка данных. При существующем аккаунте с таким же логином, пользователь видит соответствующее сообщение о том, что логин занят. В другом случае создается новый пользователь, запись сохраняется в БД. После успешной регистрации клиенту выдается access token, и его перенаправляет на страницу авторизованного пользователя (рис. 3.1).

Для входа в систему пользователь вводит свои логин и пароль на странице авторизации. Далее осуществляется валидация данных. Если логин или пароль указаны неверны, то пользователь получает соответствующее сообщение о некорректности данных. В другом случае пользователь получает токен доступа и перенаправляется на страницу авторизованного пользователя. Токен доступа хранится на клиентской части (рис. 3.2).

В случае, когда срок жизни токена доступа истечет, пользователь автоматически будет перенаправлен на страницу авторизации. При повторном входе пользователю выдается новый токен доступа, который хранится в cookie браузера.

Для выхода из аккаунта на клиентской части реализован процесс удаления токена доступа из cookie. После удаления токена пользователь перенаправляется на страницу авторизации.

3.1.1. Реализация алгоритма

Схема «Регистрация пользователя»

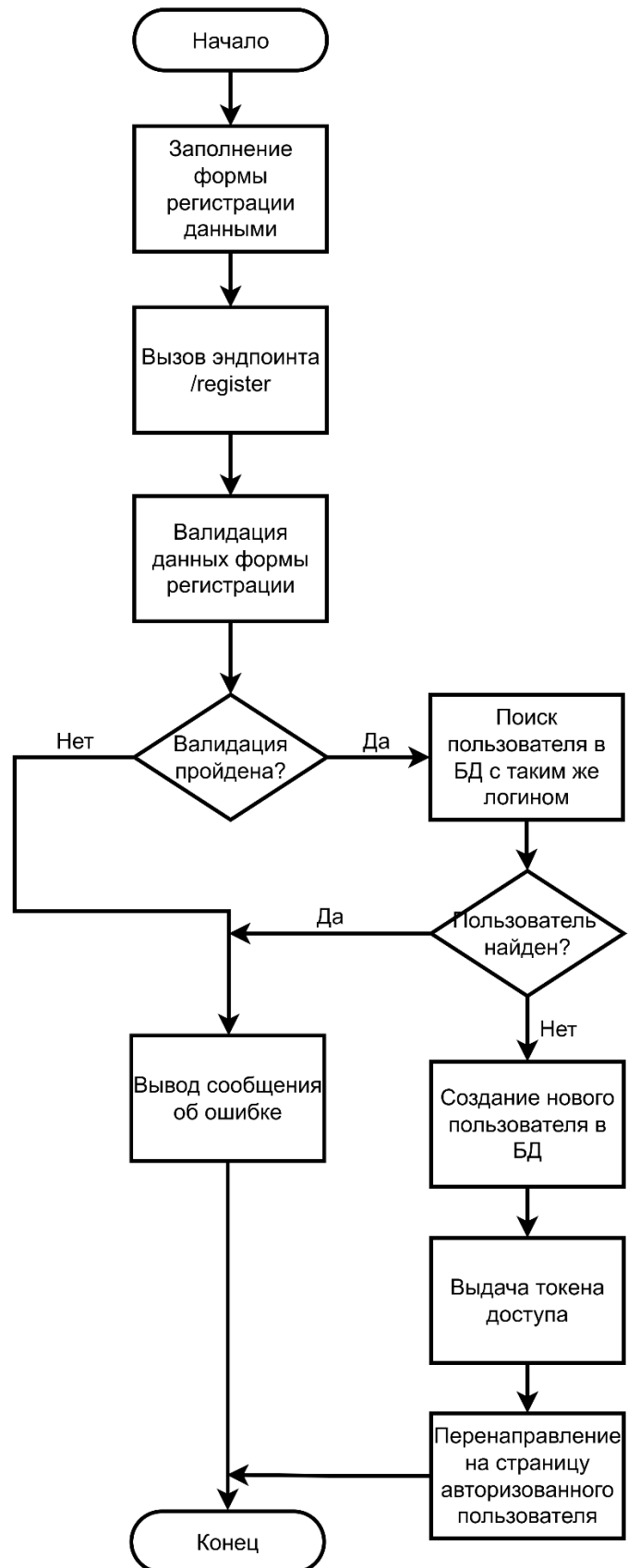


Рис. 3.1

Схема «Аутентификация пользователя»

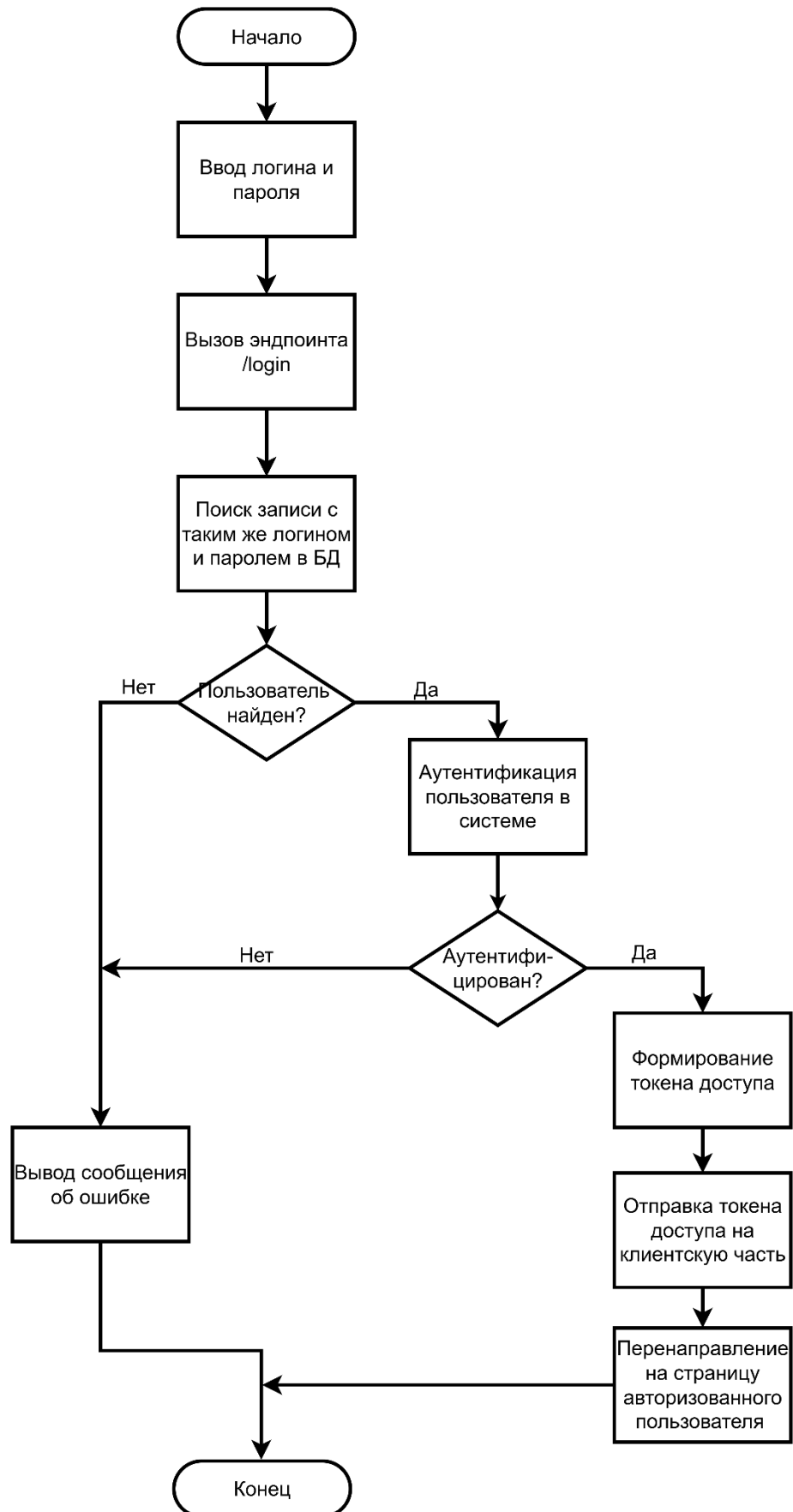


Рис. 3.2

3.1.2. Реализация классов

Для описания класса пользователя служит класс `User`. Он имеет следующие поля:

- `login: str` – поле для логина;
- `name: str` – поле для имени;
- `surname: str` – поле для фамилии;
- `patronymic: str` – поле для отчества;
- `birth_date: datetime.date` – поле для даты рождения;
- `gender: Gender` – поле для указания пола пользователя.

Класс `AuthService` предназначен для бизнес-логики аутентификации и регистрации пользователей.

Поля класса:

- `jwt_auth: JWTAuth` – сервис для генерации JWT-токенов.

Методы класса:

`register(user: UserRegisterForm, db: Session)` – регистрация нового пользователя:

- проверяет, не занят ли логин;
- создает хеш пароля;
- сохраняет пользователя в БД через репозиторий;
- генерирует JWT-токен с `payload (id, role_id)`;
- возвращает токен.

`login(user: UserLoginForm, db: Session)` – авторизация пользователя:

- проверяет существование логина;
- проверяет корректность пароля;
- генерирует JWT-токен с `payload (id, role_id)`;
- возвращает токен.

Класс `JWTAuth` необходим для работы с JWT-токенами.

Поля класса:

– config: JWTConfig – конфигурация JWT (секретный ключ, алгоритм, время жизни токена).

Методы класса:

generate_token(payload: dict) – генерирует JWT-токен:

– добавляет expiration timestamp (текущее время + TTL);

– кодирует payload с использованием секретного ключа и алгоритма;

– verify_token(token) – верифицирует JWT-токен:

– декодирует токен с использованием секретного ключа;

– возвращает payload токена.

Модуль AuthRouter содержит два контроллера (эндпоинта):

– POST /register: регистрация нового пользователя, установка cookie с токеном;

– POST /login: вход пользователя, установка cookie с токеном.

3.1.3. Описание контрольного примера

Регистрация нового пользователя

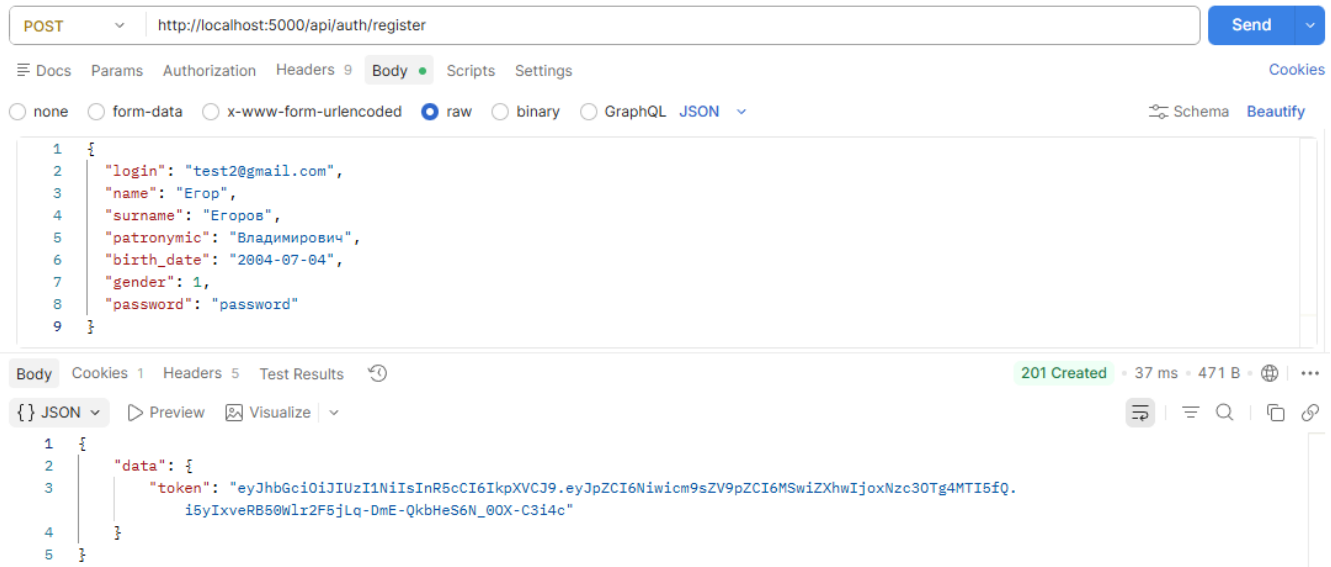


Рис. 3.3

Вход в систему

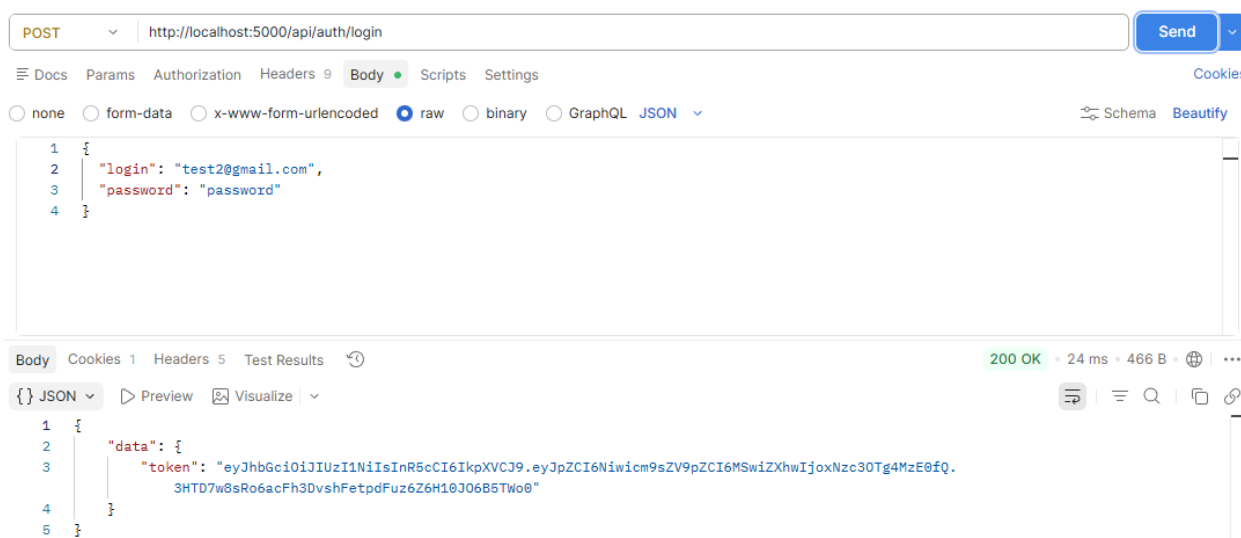


Рис. 3.4

3.2. Реализация прецедента «Регистрация участника на соревнование»

Регистрация участника на соревнование возможна только при условии, что в системе существует хотя бы одно текущее соревнование. Актуальные соревнования отображаются на общедоступной странице «Соревнования», где пользователь может ознакомиться с датами, местом проведения, регламентом и требованиями к участникам. При этом следует учитывать, что создавать соревнования могут только те пользователи, чьи учётные записи имеют роль не ниже организатора. Без достаточных прав доступа создание соревнований невозможно, что гарантирует ответственное управление спортивными мероприятиями.

Для подачи заявки на участие пользователь должен пройти аутентификацию в системе — неавторизованные посетители видят только общую информацию о соревнованиях, но не могут заполнять формы. После входа в аккаунт пользователю становится доступна страница регистрации, где все личные данные, включая фамилию, имя и отчество, автоматически подтягиваются из профиля. Участник не может редактировать свои ФИО в заявке, поскольку они считаются подтверждёнными на этапе регистрации аккаунта. В форме регистрации участник самостоятельно выбирает из

выпадающего списка свою возрастную категорию, а затем указывает весовую категорию. Также требуется указать спортивное звание, если оно имеется, — в противном случае следует выбрать вариант «без разряда». Обязательным является и название команды, которую представляет спортсмен; для индивидуальных участников допускается указать в этом поле «личное участие», однако пустым оно оставаться не может.

3.2.1. Реализация алгоритма

Схема «Подача заявки на участие в соревнованиях»

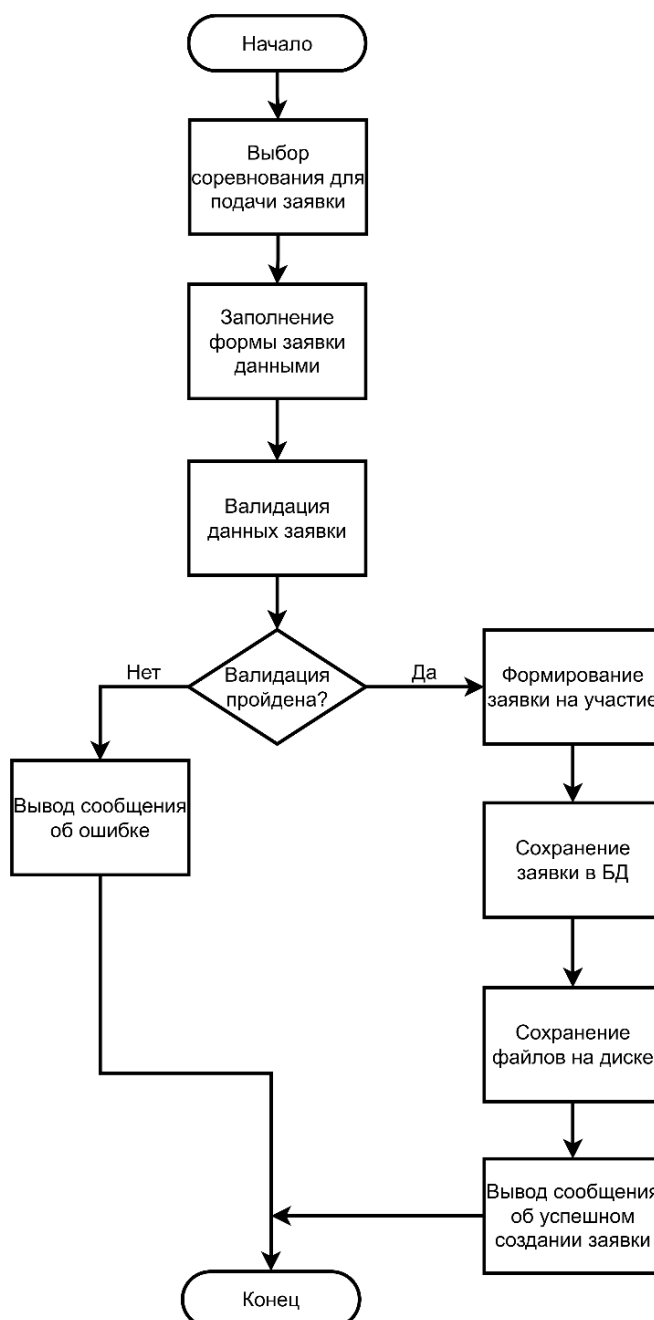


Рис. 3.5

3.2.2. Реализация классов

Класс `ApplicationSchema` представляет собой схему для описания структуры заявки. Он содержит поля:

- `id: Optional[int]` - идентификатор заявки;
- `age_category_id: int` - ID возрастной категории;
- `weight_category_id: int` - ID весовой категории;
- `user_id: Optional[int]` - ID пользователя;
- `competition_id: Optional[int]` - ID соревнования;
- `weight: Optional[float]` - вес участника;
- `rank_id: int` - ID разряда/спортивного звания;
- `team: str` - название команды;
- `created_at: Optional[datetime]` - дата и время создания;
- `update_at: Optional[datetime]` - дата и время последнего обновления;
- `status: Optional[StatusCode]` - статус заявки (например, `pending`, `approved`, `rejected`);
- `files: Optional[list[FileSchema]]` - список прикрепленных файлов (документов) к заявке;
- `full_name: Optional[str]` - полное ФИО участника (формируется как "Фамилия Имя Отчество").

Класс `FileSchema` описывает структуру объекта Файл. В нем содержатся следующие поля:

- `id: Optional[int]` - уникальный идентификатор файла в БД;
- `name: str` - оригинальное имя файла;
- `path: str` - путь к файлу на сервере;
- `application_id: int` - идентификатор заявки, к которой прикреплен данный файл.

Классы `ApplicationRepository` и `FileRepository` нужны для создания слоя между роутерами и БД. Они взаимодействуют с БД, используя CRUD-операции.

Модуль `ApplicationRouter` содержит эндпоинт с POST-запросом `/[{competition_id}]/application`, к которому осуществляется запрос с клиента. К данному эндпоинту доступ имеют только зарегистрированные пользователи.

3.2.3. Описание контрольного примера

Подача заявки на участие в соревновании. Серверная часть

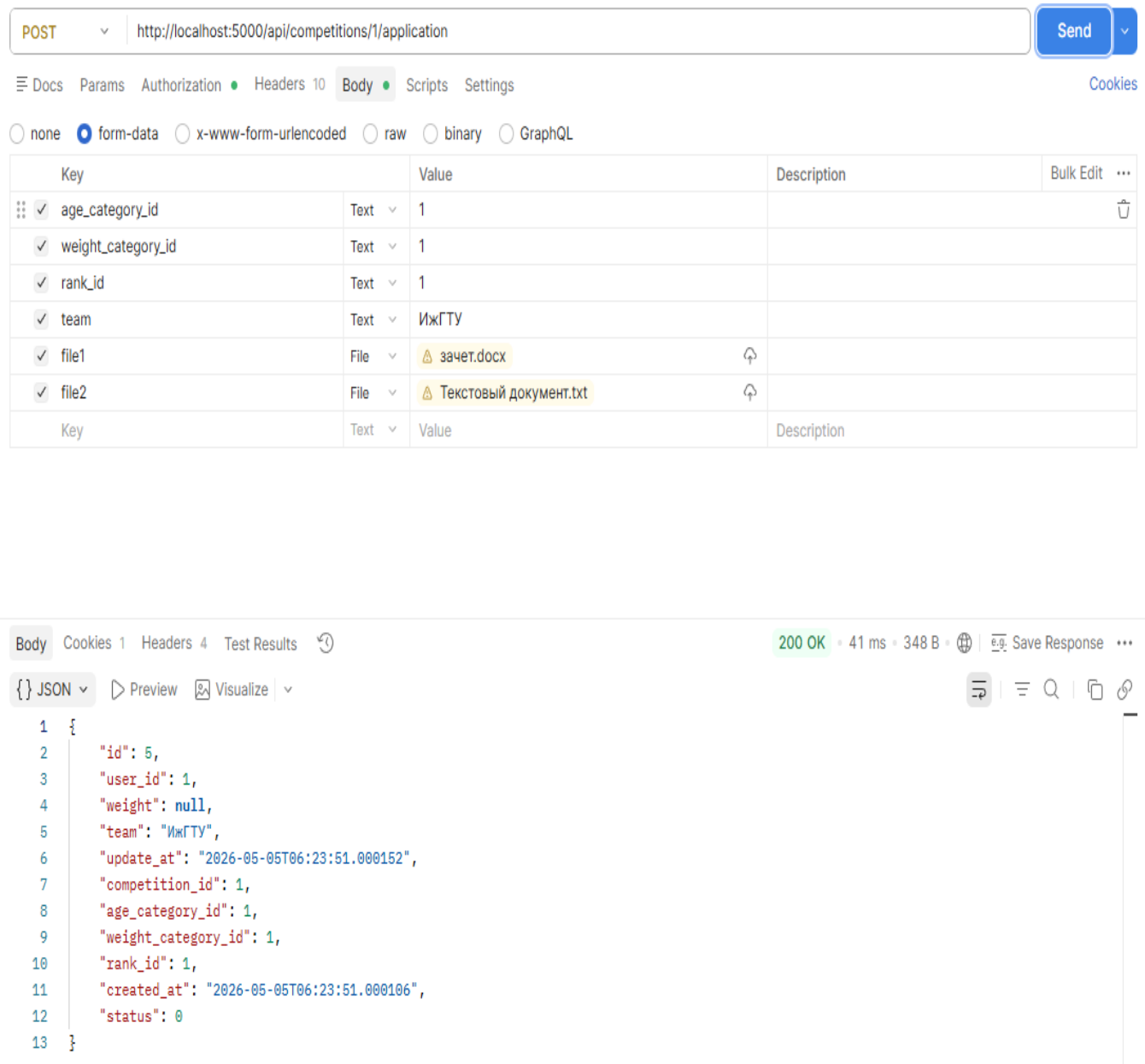


Рис. 3.6

Подача заявки на участие в соревновании. Форма в клиентской части

Подача заявки на соревнование

Возрастная категория:

Юниоры (19-21 лет) (2005 г.р. - 2007 г.р.) ▼

Весовая категория:

85 кг (80 - 85 кг) ▼

Спортивное звание:

КМС ▼

Команда:

ИжГТУ

Копия паспорта:

Выберите файл копия паспорта.pdf

Файл страхового полиса:

Выберите файл страховой полис.pdf

☒ Я даю согласие на обработку моих персональных данных

Подать заявку

Рис. 3.7

3.3. Реализация прецедента «Проверка заявки организатором соревнований»

У пользователя с ролью «Организатор» есть возможность просматривать заявки участников соревнований на отдельной странице. Это необходимо для проверки введенных данных участником. Нужно проверить паспортные данные, а также текущую медицинскую страховку. После проверки организатор меняет статус заявки на «Принята» или «Отказано» в зависимости от корректности данных. Для того, чтобы сообщить участнику о его статусе заявки, отправляется письмо на электронную почту пользователя, которую он указал при регистрации в поле «Логин».

3.3.1. Реализация алгоритма

Схема «Проверка заявки организатором соревнования»

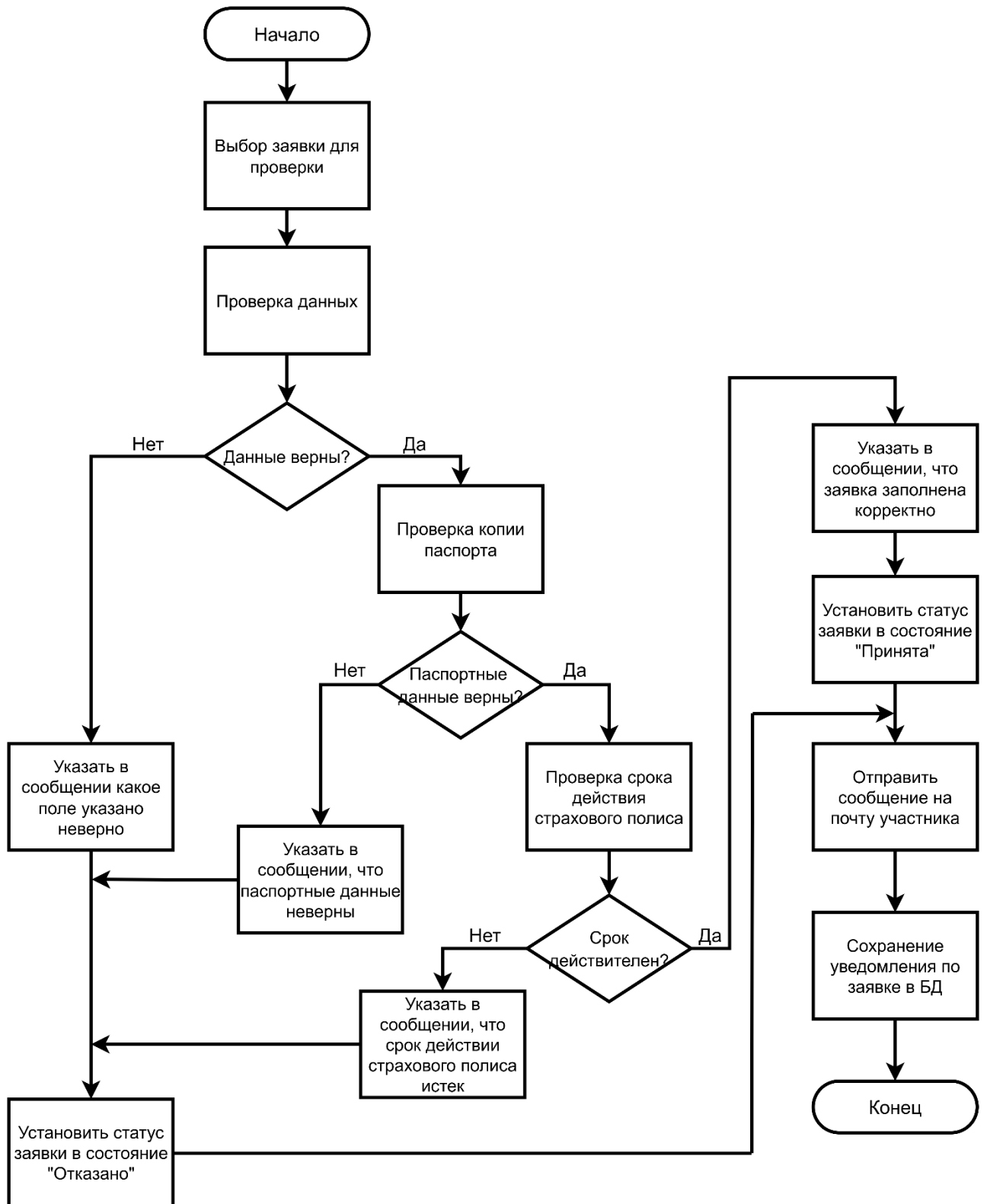


Рис. 3.8

3.3.2. Реализация классов

Классы для описания заявки описаны уже в п. 3.2.2. Класс `NotificationModel` используется для описания структуры уведомления по заявке. Он содержит следующие поля:

- `id: int` - уникальный идентификатор уведомления;
- `producer_id: int` - идентификатор отправителя заявки;
- `application_id: int` идентификатор заявки, для которого отправляется уведомление;
- `text: str` текст уведомления. Может содержать дополнительную информацию для участника соревнований.

Для реализации отправки уведомления на почту участника существует класс `EmailSender`. Основные поля класса:

- `smtp_server: str` - SMTP-сервер для отправки сообщения;
- `smtp_port: int` - SMTP-порт;
- `email: str` - почта, с которой отправляются уведомления на почту участника;
- `password: str` - пароль от почты, с которой отправляются сообщения.

Метод класса `send_text_email()` позволяет асинхронно отправлять сообщения с текстом на почту участника.

Для изменения статуса заявки есть PATCH-запрос `/competition_id/application/{application_id}`, который обновляет статус заявки в зависимости от корректности введенных данных.

Для отправки запроса с клиентской части существует POST-запрос `/notifications/{application_id}`, который сохраняет уведомление в БД, а затем асинхронно отправляет сообщение на почту участника.

3.3.3. Описание контрольного примера

Список со всеми заявками на соревнование

Все заявки участников

Соревнование: Чемпионат УР по армрестлингу

ФИО участника: Егоров Егор Владимирович

Возрастная категория: Юниоры (19-21 лет)

Весовая категория: 85 кг

Команда: ИжГТУ

Создана: 05:49, 08 мая 2026

Обновлена: 05:49, 08 мая 2026

Статус: Неизвестно

Прикрепленные файлы:

копия паспорта.pdf

Скачать

страховой полис.pdf

Скачать

Изменить статус:

В процессе

Рис. 3.9

Окно для отправки уведомления по заявке

Все заявки участников

Соревнование: Чемпионат УР по армрестлингу

ФИО участника: Егоров Егор Владимирович

Возрастная категория: Юниоры (19-21 лет)

Весовая категория: 85 кг

Команда: ИжГТУ

Создана: 05:49, 08 мая 2026

Обновлена: 05:49, 08 мая 2026

Статус: Неизвестно

Прикрепленные файлы:

копия паспорта.pdf

Скачать

страховой полис.pdf

Скачать

Изменить статус:

В процессе

Отправка уведомления на почту

Новый статус: Отказано

Сообщение:

Срок действия страхового полиса истек!

Отправить

Отмена

Рис. 3.10

Сообщение от организатора на почте участника

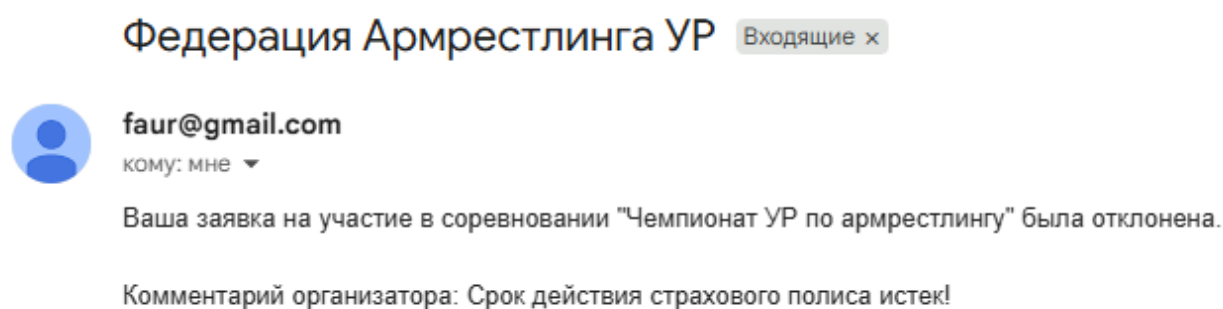


Рис. 3.11

3.4. Реализация прецедента «Просмотр стартовых списков»

Для каждого соревнования автоматически создается стартовый список. Он включает в себя список, который группируется по возрастным категориям, а внутри возрастных категорий – по весовым. Стартовый список содержит информацию о фамилии, имени, разряде и команде участников. В стартовый список определяются лишь те спортсмены, заявка которых подтверждена организатором соревнований. По стартовому списку можно определить предварительное количество спортсменов на соревновании. Для участников соревнований этот список позволяет выявить сильнейших оппонентов.

На странице «Соревнования» на каждой карточке мероприятия содержится кнопка «Стартовый список». При нажатии на нее пользователя перенаправляет на страницу со стартовым списком конкретного соревнования. Она содержит список, состоящий из нескольких уровней.

Первый уровень – возрастная категория. Поскольку в одной возрастной категории могут быть несколько весовых категорий, она имеет первый уровень.

Второй уровень – весовая категория. Необходима для составления категорий, в рамках которой участники будут находиться в равных условиях.

Третий уровень – список самих участников.

3.4.1. Реализация алгоритма

Схема «Просмотр стартовых списков»

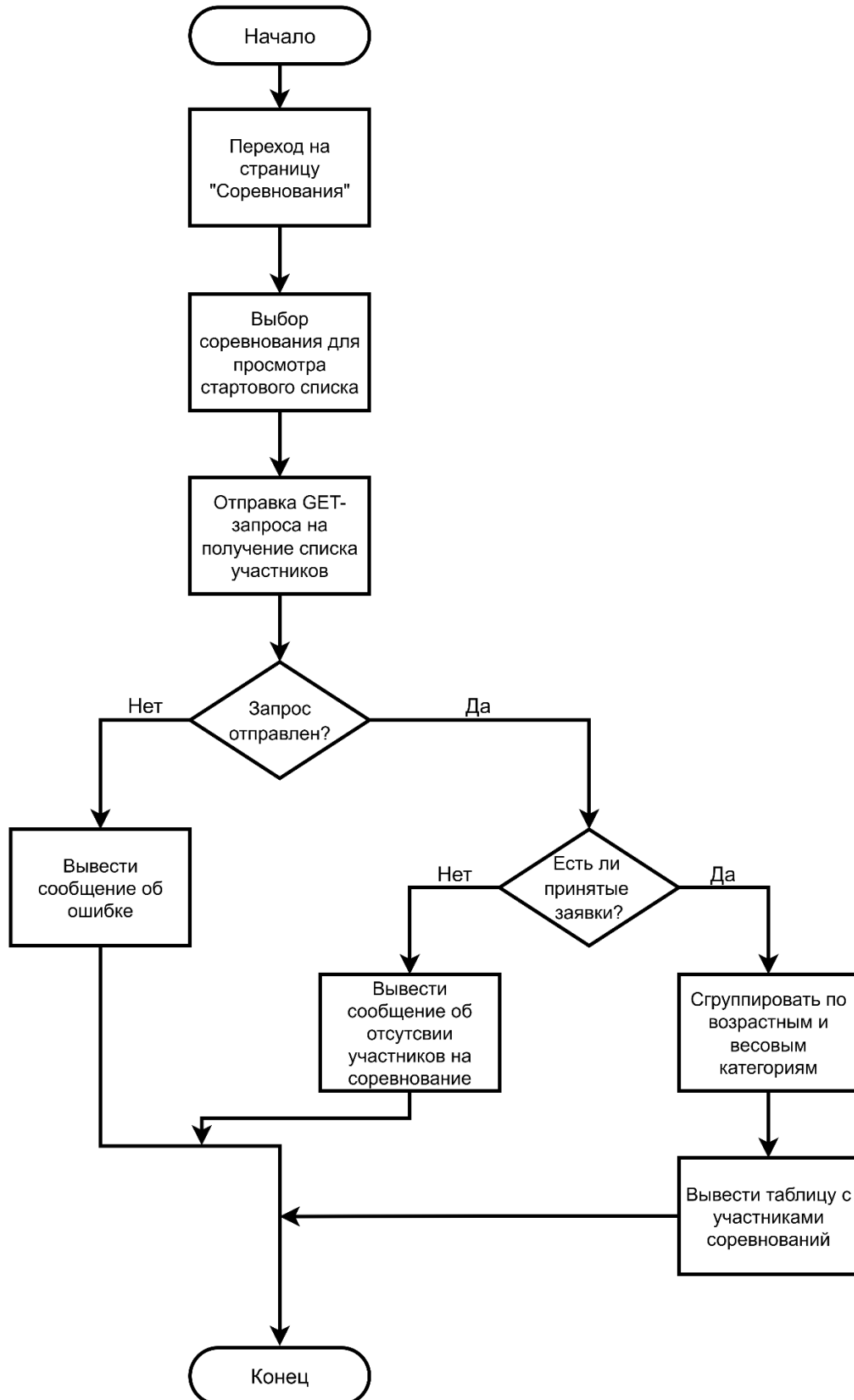


Рис. 3.12

3.4.2. Реализация классов

Для получения сгруппированного списка участников используется класс `CompetitionRepository`. В нем содержится метод `get_startlist_for_competition(competition_id)`, который на вход принимает идентификатор соревнования, для которого требуется вернуть стартовый список участников. Для отправки GET-запроса `competitions/{competition_id}/list` реализован модуль `CompetitionRouter`, который содержит контроллер для вызова эндпоинта.

3.4.3. Описание контрольного примера

Страница со стартовым списком

[← Назад](#)
Стартовый список: Чемпионат УР по армрестлингу
 Всего участников: 2

Юниоры (16-18 лет)				
70 кг 1 участ.				
№	ФАМИЛИЯ	ИМЯ	РАЗРЯД	КОМАНДА
1	Тронин	Игорь	1 раз.	УдГАУ

Юниоры (19-21 лет)				
85 кг 1 участ.				
№	ФАМИЛИЯ	ИМЯ	РАЗРЯД	КОМАНДА
1	Егоров	Егор	КМС	ИжГТУ

Рис. 3.13

3.5. Реализация прецедента «Создание турнирной сетки»

Турнирная сетка создается для каждой возрастной и весовой категории на каждую руку (левая, правая). Правом для создания турнирной сетки обладает организатор соревнования. К соревнованию допускаются лишь те спортсмены, заявка которых одобрена и подтверждена организатором соревнования.

Перед началом турнира участники проходят процедуру взвешивания, и одновременно с этим организатор вводит эти данные в систему. Также

предусмотрена возможность регистрации участника, который не подавал предварительную заявку.

3.5.1. Реализация алгоритма

Схема «Создание турнирной сетки»

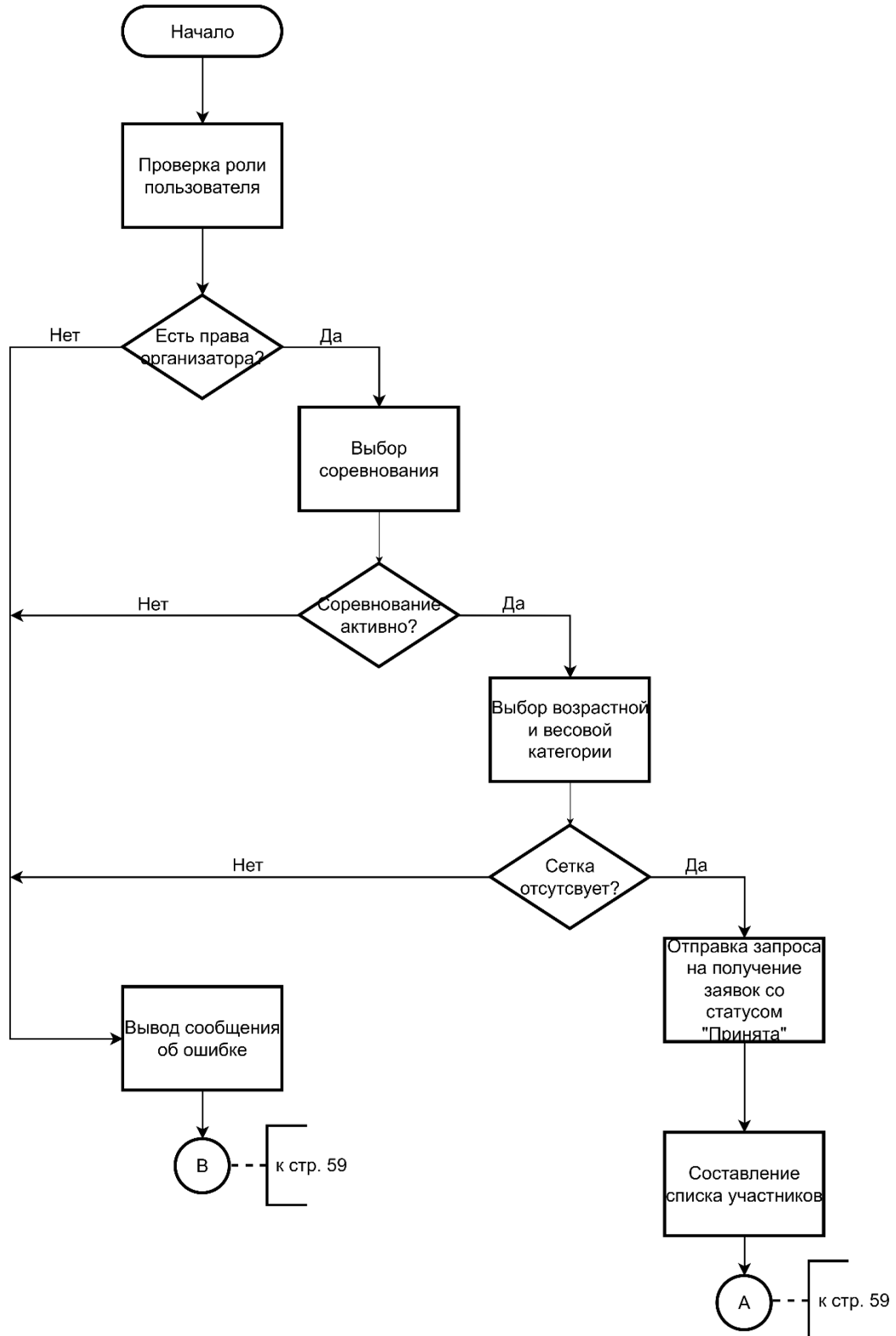


Рис. 3.14



Продолжение рис. 3.14

3.5.2. Реализация классов

Класс заявки участника был описан уже в п. 3.2.2.

Класс для описания турнира выглядит следующим образом:

- id: int - уникальный идентификатор турнирной сетки;
- tournament_id: int - уникальный строковый идентификатор турнирной сетки;
- competition_id: int - ID соревнования, в рамках которого проводится турнирная сетка;
- organizer_id: int - ID пользователя-организатора, который создал этот турнир;
- hand: str - рука, для которой проводится турнирная сетка (левая или правая);
- age_category_id: int - ID возрастной категории;
- weight_category_id: int - ID весовой категории;
- participants_data: str - JSON-строка, в которой хранятся все участники турнира;
- state_json: str - JSON-строка с полным состоянием турнира: текущий раунд, какие матчи уже сыграны, кто победил, какой матч сейчас ожидает выбора победителя;
- is_active: bool - флаг: True - турнир ещё не завершён, False - завершён или данные сохранены в историю;
- created_at: datetime - дата и время создания записи;
- updated_at: datetime - дата и время последнего обновления записи.

Для создания турнирной сетки используется POST-запрос `/[{competition_id}]/start` из модуля TournamentRouter, который получает список всех подтвержденных заявок.

- выигравший в верхней сетке переходит в следующий тур в ней же;
- проигравший в верхней сетке переходит в следующий тур в нижней сетке;
- выигравший в нижней сетке переходит в следующий тур в ней же;
- проигравший в нижней сетке выбывает из турнира.

3.6.1. Реализация алгоритма

Схема «Ведение турнирной сетки»

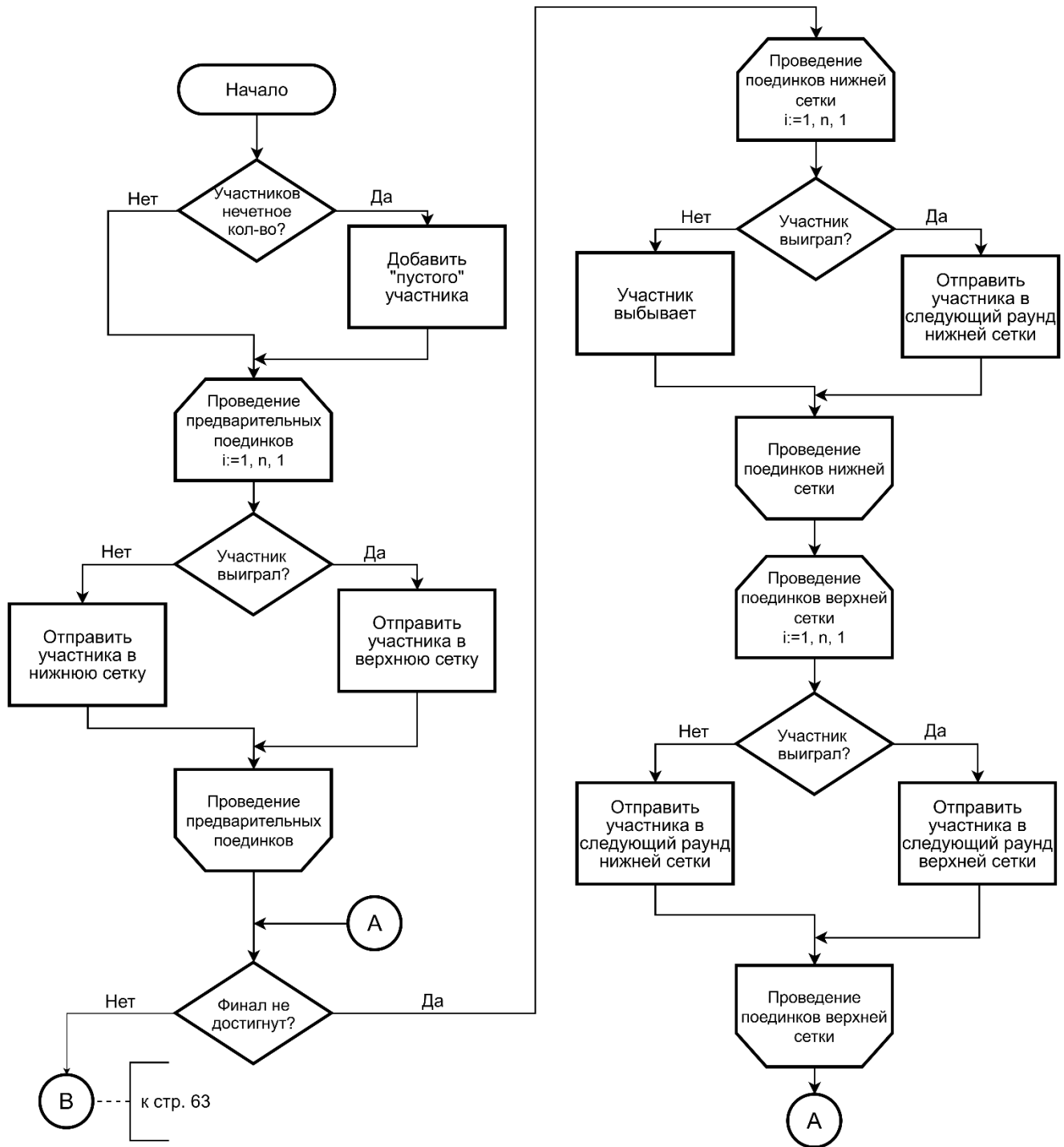
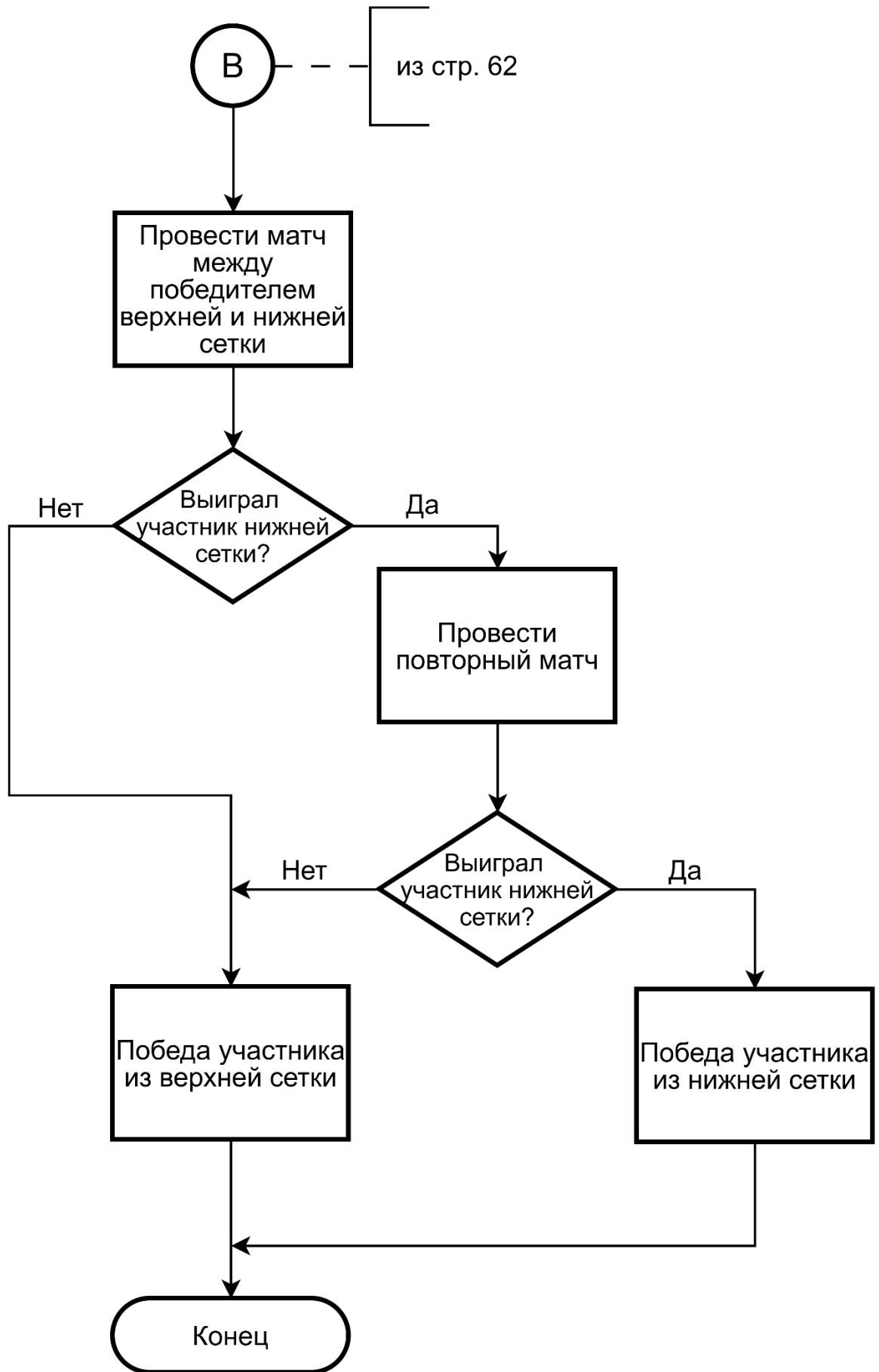


Рис. 3.16



Продолжение рис. 3.16

3.6.2. Реализация классов

Для описания матча используется класс `MatchSchema`. Он содержит следующие поля:

- `id: int` - идентификатор матча;
- `bracket_id: int` – идентификатор турнирной сетки;
- `round_number: int` - номер раунда;
- `match_number: int` - номер матча в раунде;
- `participant1_id: Optional[int]` - идентификатор первого участника;
- `participant2_id: Optional[int]` - идентификатор второго участника;
- `winner_id: Optional[int]` - идентификатор победителя;
- `loser1_id: Optional[int]` - идентификатор проигравшего (для нижней сетки);
- `participant1_score: int` - счёт первого участника (по умолчанию 0);
- `participant2_score: int` - счёт второго участника (по умолчанию 0);
- `next_winner_match_id: Optional[int]` - идентификатор следующего матча в верхней сетке;
- `next_loser_match_id: Optional[int]` - идентификатор следующего матча в нижней сетке;
- `status: MatchStatus` - статус матча.

Класс `Tournament` описывает логику ведения турнирной сетки. После подключения `WebSocket` запускается `run_tournament`, который последовательно создаёт раунды для `winners` и `losers`. Для каждого матча организатору присылается запрос через `WebSocket` (`match`), он выбирает победителя → ответ `choice` → обрабатывается в `make_choice`. Все матчи сохраняются в БД, состояние сохраняется после каждого действия (`save_state_to_db`). Организатор может нажать кнопку «Сохранить результаты» – тогда вызываются `calculate_placements` и `save_results_to_db`, итоговые места записываются в протокол.

3.6.3. Описание контрольного примера

Страница ведения турнирной сетки

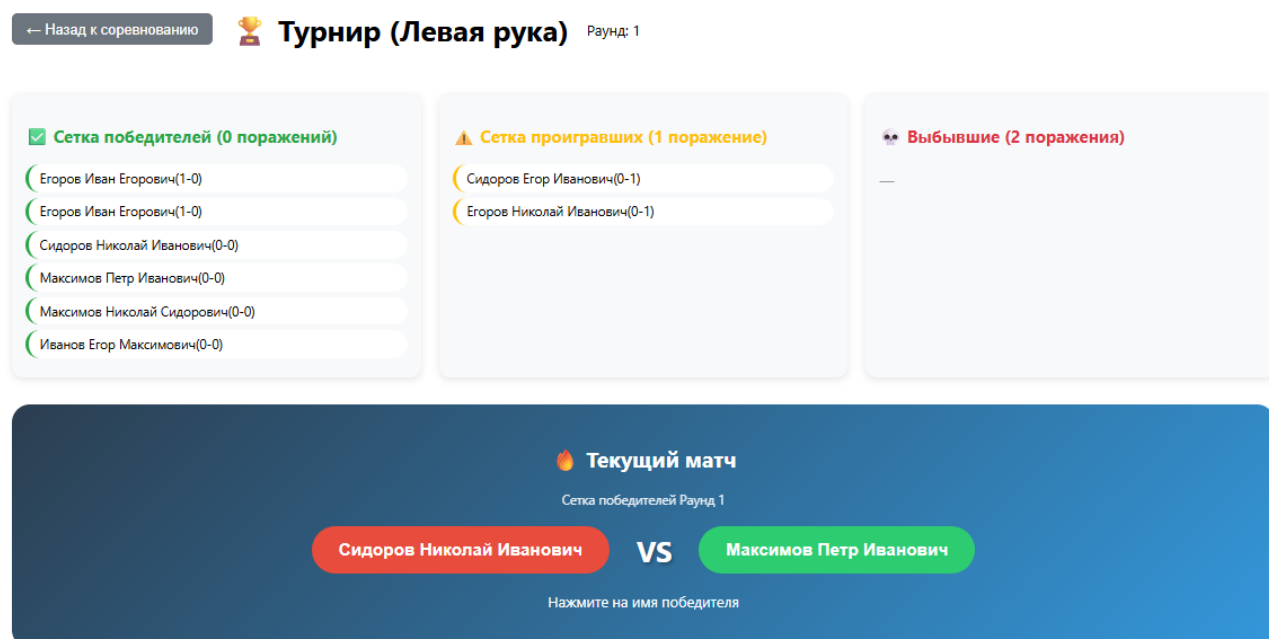


Рис. 3.17

3.7. Реализация прецедента «Составление итогового протокола соревнования»

По итогам соревнований составляется итоговый протокол. В нем отражаются результаты участников по всем возрастным и весовым категориям. В протоколе представлены имя участника и его результаты на левой, правой руках и общее место в двоеборье. Также в протоколе содержится информация о самих соревнованиях, которая включает в себя ФИО организатора, место и дата проведения соревнований.

Очки назначаются за место каждой руки, а общее место спортсмена вычисляется по сумме мест. Если сумма очков одинакова – преимущество отдается участнику с наименьшим весом.

Очки назначаются следующим образом: 1 место – 25 очка, 2 место – 17 очка, 3 место – 9 очка, 4 место – 5 очка, 5 место – 3 очка, 6 место - 2 очка.

3.7.1. Реализация алгоритма

Схема «Составление итогового протокола соревнований»

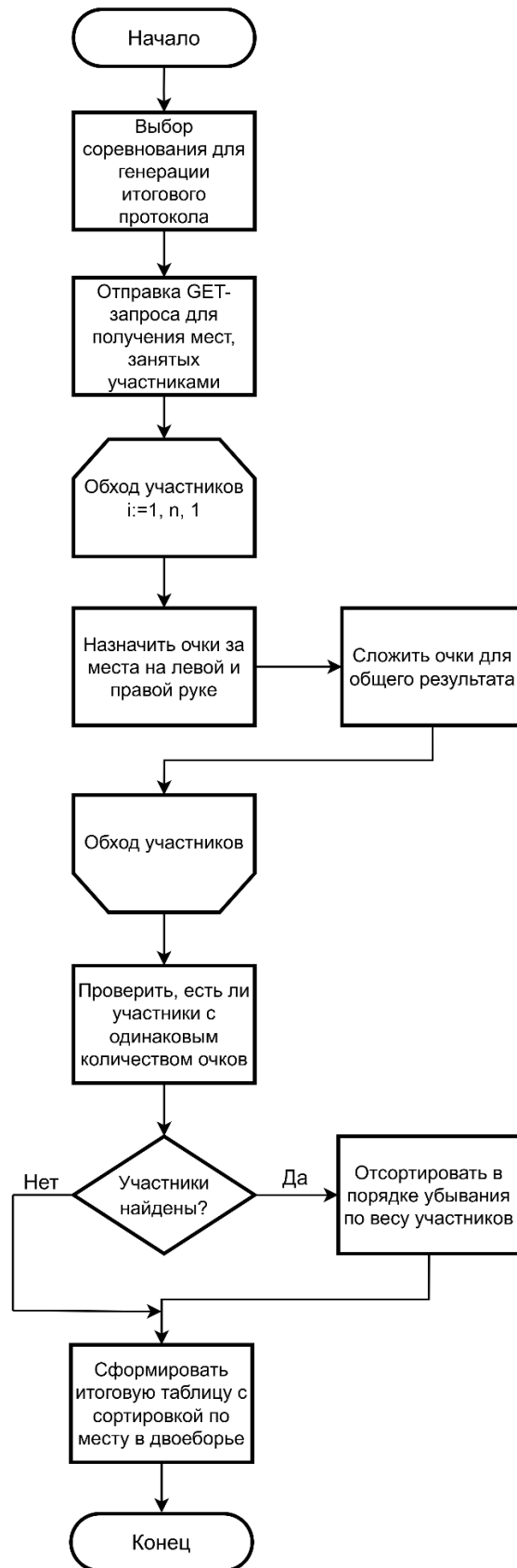


Рис. 3.18

3.7.2. Реализация классов

Для описания результатов используется класс ResultsSchema. Он содержит следующие поля:

- id: уникальный идентификатор результата;
- participant_id: внешний ключ для связи участника и его результата;
- left_hand_place: место, занятое левой рукой;
- right_hand_place: место, занятое правой рукой.

Для составления общего итога соревнований используется класс ResultRepository. Метод класса get_results_for_competition(competition_id: int) обращается к БД для получения мест участников по возрастным и весовым категориям для указанного соревнования.

3.7.3. Описание контрольного примера

Страница с итоговым протоколом соревнований

Юниоры (19-21 лет)										
85 кг										10 участн.
МЕСТО ПО "ДВОЕБОРЬЮ"	ФАМИЛИЯ, ИМЯ, ОТЧЕСТВО	ПОЛ, ДАТА РОЖДЕНИЯ	СПОРТИВНОЕ ЗВАНИЕ	КОМАНДА	МЕСТО В БОРЬБЕ ЛЕВОЙ РУКОЙ	ОЧКИ	МЕСТО В БОРЬБЕ ПРАВОЙ РУКОЙ	ОЧКИ	СУММА ОЧКОВ	ВЕС СПОРТСМЕНА
1	Максимов Николай Петрович	муж, 04.07.2004	1 раз.	ИжГТУ	2	17	1	25	42	83.50
2	Егоров Максим Сидорович	муж, 04.07.2004	1 юн. раз	ИжГТУ	1	25	10	0	25	81.79
3	Сидоров Егор Сидорович	муж, 04.07.2004	3 юн. раз.	ИжГТУ	4	5	2	17	22	84.23
4	Сидоров Иван Егорович	муж, 04.07.2004	1 юн. раз	ИжГТУ	5	3	3	9	12	81.19
5	Иванов Петр Сидорович	муж, 04.07.2004	1 юн. раз	ИжГТУ	3	9	7	0	9	81.71
6	Иванов Николай Петрович	муж, 04.07.2004	2 юн. раз.	ИжГТУ	9	0	4	5	5	81.38

Рис. 3.19

3.8. Реализация прецедента «Редактирование профиля»

При регистрации пользователь может указать неверные данные или допустить ошибку в своих ФИО. Для этого предусмотрен функционал редактирования профиля. Пользователю доступна отдельная страница для изменения данных своего профиля.

3.8.1. Реализация алгоритма

Схема «Редактирование профиля»

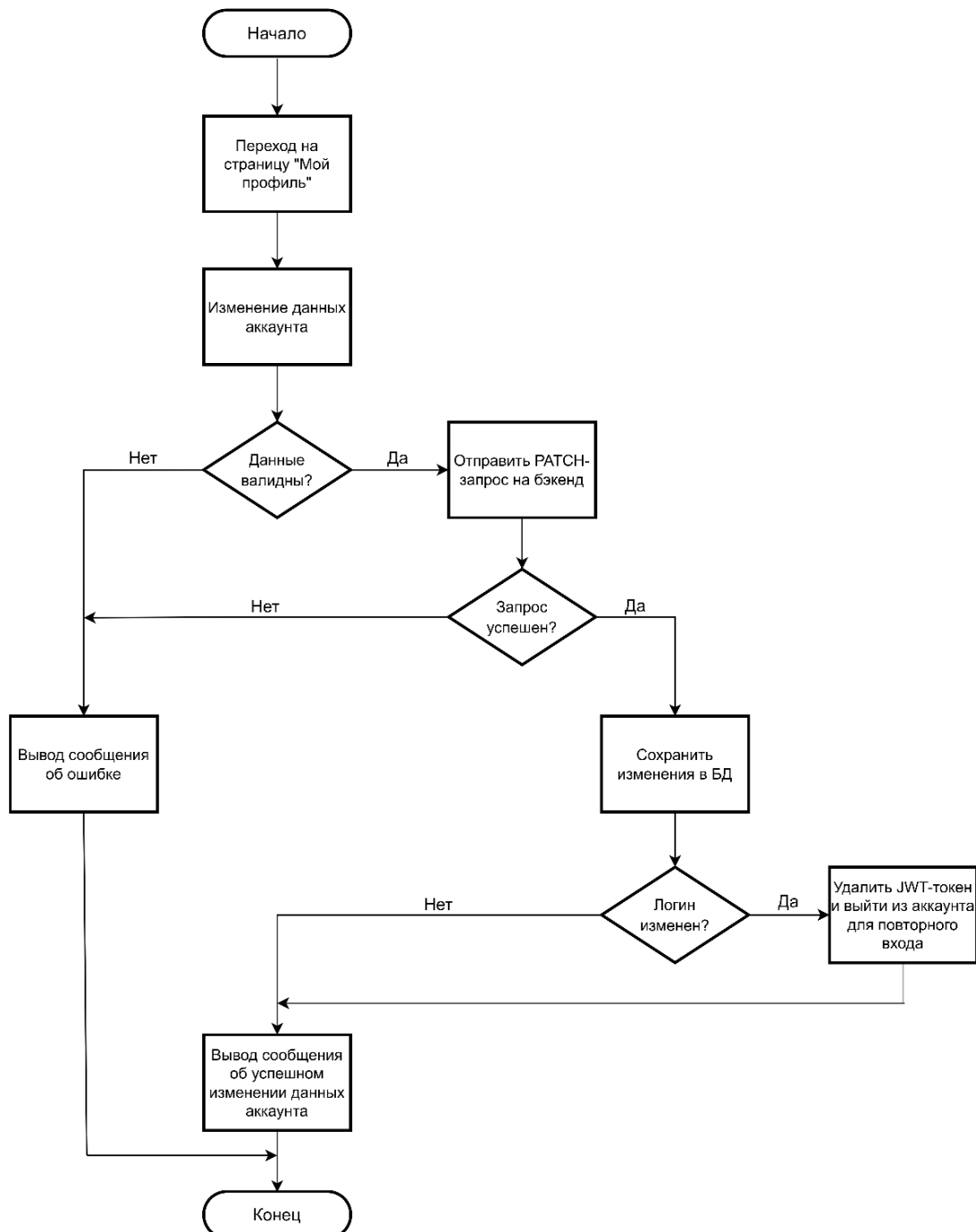


Рис. 3.20

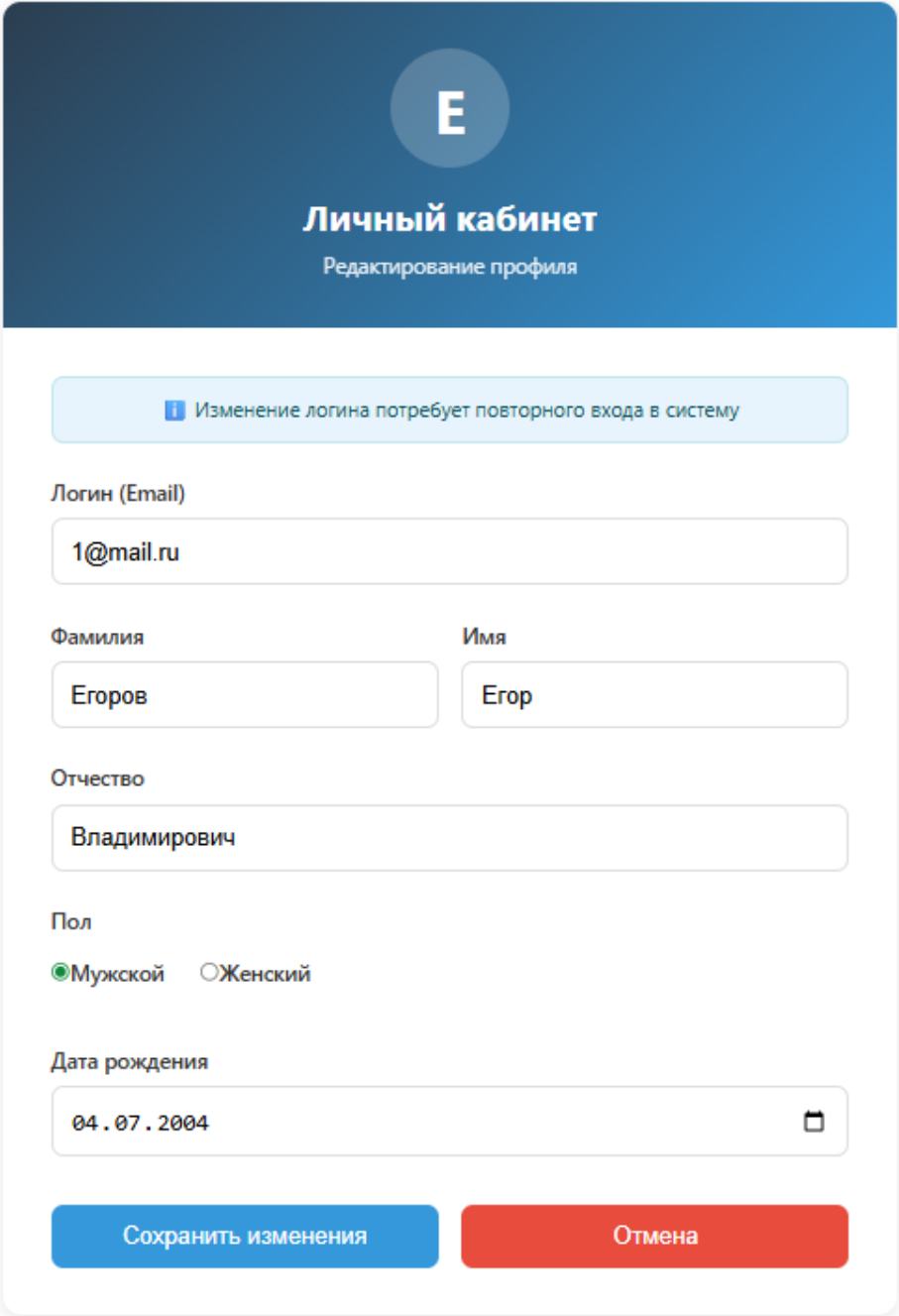
3.8.2. Реализация классов

Для описания класса пользователя используется класс `User`, который описан в п. 3.1.2.

Модуль `UserRouter` используется для отправки PATCH-запроса `/api/users/{id}` для изменения данных аккаунта.

3.8.3. Описание контрольного примера

Страница редактирования информации профиля



Е

Личный кабинет
Редактирование профиля

И Изменение логина потребует повторного входа в систему

Логин (Email)
1@mail.ru

Фамилия
Егоров

Имя
Егор

Отчество
Владимирович

Пол
☒ Мужской ☐ Женский

Дата рождения
04.07.2004

Сохранить изменения Отмена

Рис. 3.21

Ответ сервера

Curl

```
curl -X 'PATCH' \
'http://localhost:5000/api/users/{id}' \
-H 'accept: application/json' \
-H 'Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MywiYm9sZV9pZCI6MSwiZXhwIjoxNzc4NTAyMTk3FQ.6vFDH4M692k48I05vE468aahJ0F5CuKZ7CeqcipFoU' \
-H 'Content-Type: application/json' \
-d '{
  "login": "i@mail.ru",
  "name": "string",
  "surname": "string",
  "patronymic": "string",
  "birth_date": "2026-05-11",
  "gender": 1
}'
```

Request URL

```
http://localhost:5000/api/users/{id}
```

Server response

Code	Details
200	Response body <pre>{ "login": "i@mail.ru", "name": "string", "surname": "string", "patronymic": "string", "birth_date": "2026-05-11", "gender": 1 }</pre>

Рис. 3.22

ЗАКЛЮЧЕНИЕ

В процессе работы над моим дипломным проектом я создал систему управления соревнованиями по армрестлингу. Она автоматизирует регистрацию участников, создание турнирных сеток, взвешивание спортсменов и фиксацию результатов.

Я использовал современные технологии, такие как FastAPI, SQLAlchemy, PostgreSQL, WebSockets и React.ts. Это позволило создать масштабируемую и расширяемую систему.

В результате получилась полнофункциональная веб-система, которая включает:

- сервер с REST API и WebSocket-соединениями для управления турнирами в реальном времени;
- базу данных с ORM;
- систему авторизации и ролевой модели;
- модуль для подачи и обработки заявок участников;
- функционал предстартового взвешивания;
- реализацию турнирной сетки с двойным выбыванием;
- автоматическое сохранение каждого матча в базе данных;
- генерацию итоговых протоколов в формате Microsoft Word;
- email-уведомления участников;
- клиентскую часть с адаптивным интерфейсом.

Система позволяет накапливать данные о соревнованиях, анализировать результаты и быстро формировать отчетную документацию. Она гибкая и легко адаптируемая под специфику различных спортивных федераций.

Практическая значимость моей работы - возможность внедрения системы в региональных федерациях армрестлинга, спортивных клубах и организациях. Система снижает нагрузку на организаторов, ускоряет

подготовку и проведение турниров, повышает прозрачность и объективность судейства.

Внедрение системы также экономически эффективно, поскольку сокращает время на обработку заявок и формирование документов, уменьшает затраты на бумажный документооборот и предотвращает ошибки.

Данный проект имеет следующие варианты для развития: мобильное приложение для участников и судей, добавление рейтинговой системы и статистики спортсменов, а также внедрение онлайн-трансляции результатов

В целом, моя система применима в современных условиях автоматизации спортивных мероприятий и может служить основой для создания единой цифровой экосистемы федерации армрестлинга.

СПИСОК ЛИТЕРАТУРЫ

- 1) Федерация армрестлинга России. - Электронный ресурс. URL: <https://armwrestling-rus.ru/> (Дата обращения: 01.05.2026)
- 2) Макконнелл С. Совершенный код. М.: Русская редакция, 2019. – 896 с.
- 3) Рамальо Л. Python. К вершинам мастерства. М.: ДМК Пресс, 2020. – 848 с.
- 4) Бэнкс А. React. Современные шаблоны для разработки приложений. СПб.: Питер, 2019. – 464 с.
- 5) Грубер М. PostgreSQL. Основы и практика. М.: ДМК Пресс, 2020. – 416 с.
- 6) Мартинес Ф. Docker для разработчиков. – СПб.: Питер, 2021. – 336 с.
- 7) Барсегян А. А. Базы данных. Проектирование и реализация, 2018. – 336 с.
- 8) FastAPI. Официальная документация. - Электронный ресурс. URL: <https://fastapi.tiangolo.com> (дата обращения: 05.05.2026)
- 9) Гусев В. С., Артемьева И. В. Основы REST API: проектирование и реализация. СПб.: Питер, 2023. – 256 с.
- 10) Поляков С. В. PostgreSQL: практическое руководство. М.: ДМК Пресс, 2021. – 464 с.
- 11) Черкасов П. Н. Основы разработки веб-приложений на React.js. СПб.: Питер, 2022. – 288 с.
- 12) Майоров А. С. Docker и контейнеризация приложений. М.: БХВ Петербург, 2021. – 384 с.

ПРИЛОЖЕНИЕ 1

ТЕКСТЫ ПРОГРАММ

П.1.1 Код модуля заявок

П.1.1.1 Код файла router.py

```

from fastapi import APIRouter, Body, UploadFile, Form, HTTPException, Query
from fastapi import Depends
from sqlalchemy.orm import Session
from sqlalchemy.sql.functions import current_user
from starlette import status

from src.auth.models import UserModel
from src.applications.models import ApplicationModel, RankModel,
WeightCategoryModel, AgeCategoryModel
from src.competitions.models import CompetitionModel
from src.competitions.weight_categories.repository import
WeightCategoryRepository
from src.competitions.age_categories.repository import AgeCategoryRepository
from src.auth.schemas import UserBase
from src.auth.repository import AuthRepository
from src.applications.schemas import ApplicationDBSchema,
ManualApplicationCreateSchema
from src.applications.repository import ApplicationRepository
from src.applications.schemas import ApplicationSchema
from src.applications.types import StatusCode
from src.auth.router import check_user_role
from src.auth.router import get_current_user
from src.auth.schemas import Role, UserInfo
from src.database.db import get_db
from src.files.repository import FileRepository
from src.files.schemas import FileSchema
from src.files.utils import save_upload_file

router = APIRouter()

router_user_applications = APIRouter()

@router_user_applications.get("/applications")
def get_user_applications(
    user_role = Depends(check_user_role),
    current_user: UserInfo = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    result = ApplicationRepository.find_all(db, **{"user_id": current_user.id})

```

```

        applications = [ApplicationSchema.model_validate(application,
from_attributes=True) for application in result]

        for application in applications:
            file_results = FileRepository.find_all(db, **{"application_id":
application.id})
            files_validate = [FileSchema.model_validate(row, from_attributes=True)
for row in file_results]
            application.files = files_validate

        return {
            "data": applications,
        }

    @router_user_applications.get("/applications/all")
    def get_user_applications(
        user_role = Depends(check_user_role(Role.ORGANIZER)),
        db: Session = Depends(get_db)
    ):
        result = ApplicationRepository.find_all(db)

        applications = [ApplicationSchema.model_validate(application,
from_attributes=True) for application in result]

        for application in applications:
            file_results = FileRepository.find_all(db, **{"application_id":
application.id})
            files_validate = [FileSchema.model_validate(row, from_attributes=True)
for row in file_results]
            application.files = files_validate
            result = AuthRepository.find_one_or_none_by_id(application.user_id, db)
            if application.user_id:
                user = UserBase.model_validate(result, from_attributes=True)
                application.full_name = f"{user.surname} {user.name}
{user.patronymic}"
            else:
                application.full_name = f"{application.surname} {application.name}
{application.patronymic}"

        return {
            "data": applications,
        }

    @router_user_applications.patch("/applications/all/{application_id}")
    def update_application_status(
        application_id: int,
        status: StatusCode = Body(),

```

```

        user_role = Depends(check_user_role(Role.ORGANIZER)),
        db: Session = Depends(get_db)
    ):
        result = ApplicationRepository.update(
            {
                "id": application_id
            },
            db,
            **{
                "status": status
            }
        )

        return result

@router.post("/{competition_id}/application")
def create_application(
    competition_id: int,
    age_category_id: int = Form(),
    weight_category_id: int = Form(),
    rank_id: int = Form(),
    team: str = Form(),
    file1: UploadFile = Form(),
    file2: UploadFile = Form(),
    user_role=Depends(check_user_role(Role.PARTICIPANT)),
    current_user: UserInfo = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    application = ApplicationDBSchema(
        competition_id=competition_id,
        age_category_id=age_category_id,
        weight_category_id=weight_category_id,
        rank_id=rank_id,
        team=team,
    )
    application.user_id = current_user.id
    application.competition_id = competition_id

    new_application = ApplicationRepository.add(db, **application.model_dump())

    files = [file1, file2]

    for file in files:
        file_path = save_upload_file(file)
        file_schema = FileSchema(
            name=file.filename,
            path=file_path,

```

```

        application_id=new_application.id,
    )

    FileRepository.add(db, **file_schema.model_dump())

    return new_application

@router.get("/{competition_id}/application")
def get_applications(
    competition_id,
    user_role = Depends(check_user_role(Role.ORGANIZER)),
    db: Session = Depends(get_db)
) -> list[ApplicationSchema]:
    result = ApplicationRepository.find_all(db, **{"competition_id": 1})

    applications = [ApplicationSchema.model_validate(application,
from_attributes=True) for application in result]

    return applications

@router.patch("/{competition_id}/application/{application_id}")
def update_application_status(
    competition_id: int,
    application_id: int,
    status: StatusCode,
    user_role = Depends(check_user_role(Role.ORGANIZER)),
    db: Session = Depends(get_db)
):
    result = ApplicationRepository.update(
        {
            "competition_id": competition_id,
            "id": application_id
        },
        db,
        **{
            "status": status
        }
    )

    return result

@router.patch("/{competition_id}/application/{application_id}/weight")
def update_application_weight(
    competition_id: int,
    application_id: int,

```

```

        weight: float = Query(..., description="Вес участника в кг", ge=0,
le=300),

        user_role=Depends(check_user_role(Role.ORGANIZER)),
        db: Session = Depends(get_db)
    ):
        application = ApplicationRepository.find_one_or_none(
            db,
            id=application_id,
            competition_id=competition_id
        )

        if not application:
            raise HTTPException(status_code=404, detail="Заявка не найдена")

        result = ApplicationRepository.update(
            {"id": application_id, "competition_id": competition_id},
            db,
            weight=weight
        )

        if result == 0:
            raise HTTPException(status_code=404, detail="Не удалось обновить вес")

        updated_application = ApplicationRepository.find_one_or_none_by_id(application_id, db)

        return {
            "data": {
                "id": application_id,
                "weight": weight
            },
            "message": "Вес успешно сохранен"
        }

    @router.get("/{competition_id}/applications/approved")
    def get_approved_applications_for_weighing(
        competition_id: int,
        user_role=Depends(check_user_role(Role.ORGANIZER)),
        db: Session = Depends(get_db)
    ):
        approved_applications = ApplicationRepository.find_all(
            db,
            competition_id=competition_id,
            status=StatusCode.APPROVED
        )

```

```

result = []
for app in approved_applications:
    if app.user_id:
        user = db.query(UserModel).filter(UserModel.id ==
app.user_id).first()
        surname = user.surname if user else app.surname
        name = user.name if user else app.name
        patronymic = user.patronymic if user else app.patronymic
    else:
        surname = app.surname
        name = app.name
        patronymic = app.patronymic

    full_name = f"{surname} {name} {patronymic or ''}".strip()
    result.append({
        "application_id": app.id,
        "full_name": full_name,
        "weight_category": db.query(WeightCategoryModel).filter(
            WeightCategoryModel.id == app.weight_category_id).first().name,
        "weight_category_id": app.weight_category_id,
        "age_category":
db.query(AgeCategoryModel).filter(AgeCategoryModel.id ==
app.age_category_id).first().name,
        "team": app.team,
        "weight": app.weight
    })
return {
    "data": result,
}

@router.patch("/{application_id}/weight-category")
def update_application_weight_category(
    application_id: int,
    new_category_id: int,
    db: Session = Depends(get_db),
    current_user=Depends(check_user_role(Role.ORGANIZER))
):
    app = ApplicationRepository.find_one_or_none_by_id(application_id, db)
    if not app:
        raise HTTPException(status_code=404, detail="Заявка не найдена")

    new_category =
WeightCategoryRepository.find_one_or_none_by_id(new_category_id, db)
    if not new_category:
        raise HTTPException(status_code=400, detail="Некорректная
весовая категория")

```

```

app.weight_category_id = new_category_id
db.commit()
db.refresh(app)

return {
    "message": "Весовая категория обновлена",
    "application_id": app.id,
    "new_category": new_category.name,
    "new_category_id": new_category.id
}

@router.delete("/{application_id}")
def delete_application(
    application_id: int,
    db: Session = Depends(get_db),
):
    ApplicationRepository.delete(db, False, **{"id": application_id})

    return {
        "data": f"Application {application_id} was deleted",
    }

@router.post("/manual", status_code=status.HTTP_201_CREATED)
def create_manual_application(
    app_data: ManualApplicationCreateSchema,
    db: Session = Depends(get_db),
    current_user = Depends(check_user_role(Role.ORGANIZER))
):
    competition = db.query(CompetitionModel).filter(CompetitionModel.id ==
app_data.competition_id).first()
    if not competition:
        raise HTTPException(status_code=404, detail="Соревнование не найдено")

    age_cat = db.query(AgeCategoryModel).filter(AgeCategoryModel.id ==
app_data.age_category_id).first()
    if not age_cat:
        raise HTTPException(status_code=400, detail="Некорректная возрастная
категория")

    weight_cat = db.query(WeightCategoryModel).filter(WeightCategoryModel.id ==
app_data.weight_category_id).first()
    if not weight_cat:
        raise HTTPException(status_code=400, detail="Некорректная весовая
категория")

    rank = db.query(RankModel).filter(RankModel.id == app_data.rank_id).first()
    if not rank:

```



```

        raise HTTPException(status_code=400, detail="Некорректное спортивное
звание")

    new_app = ApplicationModel(
        competition_id=app_data.competition_id,
        age_category_id=app_data.age_category_id,
        weight_category_id=app_data.weight_category_id,
        rank_id=app_data.rank_id,
        team=app_data.team,
        weight=app_data.weight,
        status=StatusCode.APPROVED,
        user_id=None,
        surname=app_data.surname,
        name=app_data.name,
        patronymic=app_data.patronymic
    )
    db.add(new_app)
    db.commit()
    db.refresh(new_app)

    return {
        "message": "Участник успешно добавлен",
        "application_id": new_app.id,
        "full_name": f"{app_data.surname} {app_data.name} {app_data.patronymic or
''}".strip()
    }

```

П.1.2 Код модуля соревнований

П.1.2.1 Код файла router.py

```

from typing import Optional

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session

from src.applications.router import router as applications_router
from src.applications.schemas import AgeCategorySchema, WeightCategorySchema
from src.applications.schemas import RankSchema
from src.auth.router import check_user_role
from src.auth.schemas import Role
from src.competitions.age_categories.repository import AgeCategoryRepository
from src.competitions.ranks.repository import RankRepository
from src.competitions.repository import CompetitionRepository
from src.competitions.schemas import CompetitionSchema,
CompetitionCategoriesSchema
from src.competitions.weight_categories.repository import
WeightCategoryRepository
from src.database.db import get_db

```

```

router = APIRouter(
    prefix="/competitions",
    tags=["competitions"]
)

router.include_router(applications_router, tags=["applications"])

@router.get('/age_categories')
def get_age_categories(
    id: Optional[int] = Query(default=None),
    db: Session = Depends(get_db)
):
    if id is None:
        params = {}
    else:
        params = {"id":id}
    result = AgeCategoryRepository.find_all(db, **params)

    age_categories = [
        AgeCategorySchema.model_validate(category, from_attributes=True) for
category in result
    ]

    return {
        "data" : age_categories,
    }

@router.get('/weight_categories')
def get_weight_categories(
    id: Optional[int] = Query(default=None),
    db: Session = Depends(get_db)
):
    if id is None:
        params = {}
    else:
        params = {"id":id}
    result = WeightCategoryRepository.find_all(db, **params)

    weight_categories = [
        WeightCategorySchema.model_validate(category, from_attributes=True) for
category in result
    ]

    return {
        "data" : weight_categories,
    }

```

```

    }

    @router.get('/ranks')
    def get_ranks(
        id: Optional[int] = Query(default=None),
        db: Session = Depends(get_db)
    ):
        if id is None:
            params = {}
        else:
            params = {"id":id}
        result = RankRepository.find_all(db, **params)

        ranks = [
            RankSchema.model_validate(rank, from_attributes=True) for rank in result
        ]

        return {
            "data" : ranks,
        }

    @router.get('/')
    def get_competitions(
        db: Session = Depends(get_db)
    ) -> dict[str, list[CompetitionSchema]]:
        result = CompetitionRepository.find_all(db)
        competitions = [
            CompetitionSchema.model_validate(competition, from_attributes=True)
            for competition in result
        ]

        return {
            "data": competitions,
        }

    @router.post('/')
    def create_competition(
        competition: CompetitionSchema,
        current_user = Depends(check_user_role(Role.ORGANIZER)),
        db: Session = Depends(get_db)
    ):
        competition.organizer_id = current_user.id
        new_competition = CompetitionRepository.add(db, **competition.model_dump())

        return new_competition

```

```

@router.get('/{competition_id}')
def get_competition_info(
    competition_id: int,
    db: Session = Depends(get_db)
) -> dict[str, CompetitionSchema]:
    result = CompetitionRepository.find_one_or_none_by_id(competition_id, db)

    if result is None:
        raise HTTPException(status_code=404, detail="Competition not found")

    competition = CompetitionSchema.model_validate(result, from_attributes=True)

    return {
        "data" : competition
    }

@router.post('/{competition_id}/category')
def add_competition_category(
    category: CompetitionCategoriesSchema,
    current_user = Depends(check_user_role(Role.ORGANIZER)),
    db: Session = Depends(get_db)
) -> dict[str, CompetitionCategoriesSchema]:
    new_category = CompetitionRepository.add_category_for_competition(category,
db)

    return {
        "data" : new_category,
    }

@router.get('/{competition_id}/list')
def get_competition_list(
    competition_id: int,
    db: Session = Depends(get_db)
):
    result = CompetitionRepository.get_startlist_for_competition(competition_id,
db)

    if result is None:
        raise HTTPException(status_code=404, detail="Competition not found")

    return {
        "data" : result
    }

```

П.1.3 Код модуля уведомлений

П.1.3.1 Код файла email_sender.py

```

import asyncio
import aiosmtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText

from src.settings import settings

class EmailSender:
    def __init__(self, smtp_server: str, smtp_port: int, email: str, password:
str):
        self.smtp_server = smtp_server
        self.smtp_port = smtp_port
        self.email = email
        self.password = password

    async def send_text_email(self, to_email: str, subject: str, body: str) ->
bool:
        try:
            msg = MIMEMultipart()
            msg['From'] = self.email
            msg['To'] = to_email
            msg['Subject'] = subject
            msg.attach(MIMEText(body, 'plain', 'utf-8'))

            server = aiosmtplib.SMTP(
                hostname=self.smtp_server,
                port=self.smtp_port,
            )

            try:
                await server.connect()
                await server.login(self.email, self.password)
                await server.send_message(msg)
            finally:
                await server.quit()

            print(f"Email успешно отправлен на {to_email}")
            return True

        except Exception as e:
            print(f"Ошибка отправки email: {e}")
            return False

email_sender = EmailSender(
    smtp_server="smtp.gmail.com",

```

```

smtp_port=587,
email=settings.email_login,
password=settings.email_password,
)

```

П.1.3.2 Код файла router.py

```

from fastapi import APIRouter, Depends, Body
from sqlalchemy.orm import Session

from src.applications.repository import ApplicationRepository
from src.auth.repository import AuthRepository
from src.database.db import get_db
from src.notifications.schemas import NotificationSchema
from src.notifications.repository import NotificationRepository
from src.auth.router import check_user_role
from src.auth.schemas import Role
from src.notifications.email_sender import email_sender

router = APIRouter(
    prefix="/notifications",
)

@router.post("/{application_id}")
async def send_competition_notification(
    application_id: int,
    text: str = Body(),
    current_user = Depends(check_user_role(Role.ORGANIZER)),
    db: Session = Depends(get_db)
):
    notification = NotificationSchema(application_id=application_id,
producer_id=current_user.id, text=text)
    new_notification = NotificationRepository.add(db,
**notification.model_dump())

    application = ApplicationRepository.find_one_or_none_by_id(application_id,
db)

    user = AuthRepository.find_one_or_none_by_id(application.user_id, db)
    user_email = user.login

    await email_sender.send_text_email(user_email, "Федерация Армрестлинга УР",
text)

    return new_notification

```

П.1.4 Код модуля результатов

П.1.4.1 Код файла service.py

```

import os
from docx import Document
from docx.shared import Inches, Pt, Cm
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.enum.table import WD_TABLE_ALIGNMENT
from docx.oxml.ns import qn
from docx.oxml import OxmlElement
from docx.enum.section import WD_ORIENTATION
from datetime import datetime
from typing import List, Dict, Any

class WordExporter:
    @staticmethod
    def set_cell_border(cell, border_size=1):
        tc = cell._tc
        tcPr = tc.get_or_add_tcPr()
        for edge in ['top', 'left', 'bottom', 'right']:
            edge_element = OxmlElement(f'w:{edge}')
            edge_element.set(qn('w:val'), 'single')
            edge_element.set(qn('w:sz'), str(border_size))
            edge_element.set(qn('w:space'), '0')
            edge_element.set(qn('w:color'), '000000')
            tcPr.append(edge_element)

    @staticmethod
    def set_page_landscape(doc):
        section = doc.sections[0]
        section.orientation = WD_ORIENTATION.LANDSCAPE
        section.page_width = Cm(29.7)
        section.page_height = Cm(21.0)
        section.top_margin = Cm(1.5)
        section.bottom_margin = Cm(1.5)
        section.left_margin = Cm(1.5)
        section.right_margin = Cm(1.5)

    @staticmethod
    def add_header(doc, competition_name: str):
        """Добавляет верхний колонтитул с названием соревнования на каждой
странице"""
        section = doc.sections[0]
        header = section.header
        header_para = header.paragraphs[0] if header.paragraphs else
header.add_paragraph()
        header_para.text = f"Соревнования \"{competition_name}\""
        header_para.alignment = WD_ALIGN_PARAGRAPH.CENTER
        header_para.style = doc.styles['Header']

```

```

        for run in header_para.runs:
            run.font.size = Pt(10)
            run.font.bold = True

    @staticmethod
    def _get_gender_string(gender):
        if gender is None:
            return ''

        if hasattr(gender, 'value'):
            return 'M' if gender.value == 1 else 'Ж'
        return str(gender)[0] if str(gender) else ''

    @staticmethod
    def _get_date_string(date_value):
        if date_value is None:
            return ''

        if hasattr(date_value, 'strftime'):
            return date_value.strftime('%d.%m.%Y')
        return str(date_value)

    @staticmethod
    def export_competition_results_detailed(
        results: List[Dict[str, Any]],
        competition_name: str,
        competition_date: str,
        competition_location: str = None,
        organizer: str = None,
        total_participants: int = None,
        male_count: int = None,
        female_count: int = None
    ) -> str:
        doc = Document()
        WordExporter.set_page_landscape(doc)
        WordExporter.add_header(doc, competition_name)

        style = doc.styles['Normal']
        style.font.name = 'Times New Roman'
        style.font.size = Pt(12)

        title = doc.add_heading('ПРОТОКОЛ', 0)
        title.alignment = WD_ALIGN_PARAGRAPH.CENTER

        doc.add_paragraph()

        p = doc.add_paragraph()
        p.alignment = WD_ALIGN_PARAGRAPH.CENTER
        run = p.add_run(f'соревнований по армрестлингу\n"{competition_name}"')

```



```

run.bold = True
run.font.size = Pt(14)

doc.add_paragraph()

info_table = doc.add_table(rows=4, cols=2)
info_table.style = 'Table Grid'
info_table.autofit = False
info_table.allow_autofit = False
info_table.columns[0].width = Cm(5)
info_table.columns[1].width = Cm(21)

info_data = [
    ('Дата проведения:', competition_date),
    ('Место проведения:', competition_location or 'не указано'),
    ('Организатор:', organizer or 'не указан'),
]
for i, (label, value) in enumerate(info_data):
    if i < len(info_table.rows):
        info_table.rows[i].cells[0].text = label
        info_table.rows[i].cells[1].text = value
        info_table.rows[i].cells[0].paragraphs[0].runs[0].font.bold =
True

        for cell in info_table.rows[i].cells:
            WordExporter.set_cell_border(cell)

    if total_participants is not None and male_count is not None and
female_count is not None:
        stats_row = info_table.add_row()
        stats_row.cells[0].text = 'Участники:'
        stats_row.cells[1].text = f'Всего: {total_participants}, мужчин:
{male_count}, женщин: {female_count}'
        stats_row.cells[0].paragraphs[0].runs[0].font.bold = True
        for cell in stats_row.cells:
            WordExporter.set_cell_border(cell)

doc.add_paragraph()

for age_group in results:
    doc.add_heading(f'{age_group["age_category"]}', level=1)
    for weight_group in age_group["weight_categories"]:
        doc.add_heading(f'Весовая категория:
{weight_group["weight_category"]}', level=2)

        table = doc.add_table(rows=1, cols=10)
        table.style = 'Table Grid'
        table.autofit = False

```

```

table.allow_autofit = False
widths = [Cm(1.2), Cm(3.5), Cm(1.5), Cm(2.5),
          Cm(1.5), Cm(1.3), Cm(1.5), Cm(1.3), Cm(1.3), Cm(1.2)]
for i, width in enumerate(widths):
    if i < len(table.columns):
        table.columns[i].width = width

headers = ['Место', 'ФИО', 'Разряд', 'Команда',
           'Левая\прука\n(место)', 'Левая\прука\n(очки)',
           'Правая\прука\n(место)', 'Правая\прука\n(очки)',
           'Сумма\почков', 'Вес, кг']
header_cells = table.rows[0].cells
for i, header in enumerate(headers):
    if i < len(header_cells):
        header_cells[i].text = header
        header_cells[i].paragraphs[0].runs[0].font.bold = True
        header_cells[i].paragraphs[0].alignment =
WD_ALIGN_PARAGRAPH.CENTER

        WordExporter.set_cell_border(header_cells[i])

participants = sorted(weight_group["participants"], key=lambda x:
x.get('place_by_two_arms', 999))
for participant in participants:
    row_cells = table.add_row().cells
    athlete_weight = participant.get('athlete_weight')
    weight_str = f"{athlete_weight}" if athlete_weight else ''
    right_hand_place = participant.get('right_hand_place', '')
    if right_hand_place is None:
        right_hand_place = ''
    left_hand_place = participant.get('left_hand_place', '')
    if left_hand_place is None:
        left_hand_place = ''

    cell_values = [
        str(participant.get('place_by_two_arms', '')),
        participant.get('full_name', ''),
        participant.get('rank', '') or '',
        participant.get('team', '') or '',
        str(left_hand_place),
        str(participant.get('left_points', 0)),
        str(right_hand_place),
        str(participant.get('right_points', 0)),
        str(participant.get('total_points', 0)),
        weight_str
    ]
    for i, value in enumerate(cell_values):
        if i < len(row_cells):

```

```

        row_cells[i].text = value
        row_cells[i].paragraphs[0].alignment =
WD_ALIGN_PARAGRAPH.CENTER

        WordExporter.set_cell_border(row_cells[i])
        doc.add_paragraph()

    doc.add_paragraph()
    doc.add_paragraph()
    signatures = doc.add_table(rows=2, cols=1)
    signatures.autofit = False
    if len(signatures.columns) >= 1:
        signatures.columns[0].width = Cm(28)
    if len(signatures.rows) >= 2:
        signatures.rows[0].cells[0].text = 'Главный судья'
        signatures.rows[1].cells[0].text = ''
        for row in signatures.rows:
            for cell in row.cells:
                for paragraph in cell.paragraphs:
                    paragraph.alignment = WD_ALIGN_PARAGRAPH.CENTER
                WordExporter.set_cell_border(cell)

    os.makedirs('exports', exist_ok=True)
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
    safe_name = competition_name.replace('/', '_').replace('\\',
'_').replace(':', '_')
    output_path = f'exports/protocol_{safe_name}_{timestamp}.docx'
    doc.save(output_path)
    return output_path

```

П.1.4.2 Код файла repository.py

```

from sqlalchemy import select, case
from sqlalchemy.orm import Session

from src.applications.types import StatusCode
from src.applications.models import AgeCategoryModel, WeightCategoryModel,
ApplicationModel, RankModel
from src.auth.models import UserModel
from src.results.models import ResultModel
from src.repository import BaseRepository

class ResultsRepository(BaseRepository):
    model = ResultModel

    @staticmethod
    def get_results_for_competition(
        competition_id: int,

```

```

        db: Session
    ):
        place_points = {
            1: 25, 2: 17, 3: 9, 4: 5, 5: 3, 6: 2
        }

        left_points_case = case(
            *[(ResultModel.left_hand_place == place, points) for place, points in
place_points.items()],
            else_=0
        )
        right_points_case = case(
            *[(ResultModel.right_hand_place == place, points) for place, points
in place_points.items()],
            else_=0
        )

        surname_case = case(
            (ApplicationModel.user_id.isnot(None), UserModel.surname),
            else_=ApplicationModel.surname
        ).label("surname")

        name_case = case(
            (ApplicationModel.user_id.isnot(None), UserModel.name),
            else_=ApplicationModel.name
        ).label("name")

        patronymic_case = case(
            (ApplicationModel.user_id.isnot(None), UserModel.patronymic),
            else_=ApplicationModel.patronymic
        ).label("patronymic")

        gender_case = case(
            (ApplicationModel.user_id.isnot(None), UserModel.gender),
            else_=None
        ).label("gender")

        birth_date_case = case(
            (ApplicationModel.user_id.isnot(None), UserModel.birth_date),
            else_=None
        ).label("birth_date")

        query = (
            select(
                AgeCategoryModel.name.label("age_category"),
                WeightCategoryModel.name.label("weight_category"),
                AgeCategoryModel.id.label("age_category_id"),

```

```

        WeightCategoryModel.id.label("weight_category_id"),
        ApplicationModel.id.label("application_id"),
        surname_case,
        name_case,
        patronymic_case,
        gender_case,
        birth_date_case,
        RankModel.name.label("rank"),
        ApplicationModel.team,
        ApplicationModel.weight.label("athlete_weight"),
        ResultModel.left_hand_place,
        left_points_case.label("left_points"),
        ResultModel.right_hand_place,
        right_points_case.label("right_points"),
        (left_points_case + right_points_case).label("total_points")
    )
    .select_from(ApplicationModel)
    .outerjoin(UserModel, ApplicationModel.user_id == UserModel.id)
    .join(WeightCategoryModel, ApplicationModel.weight_category_id ==
WeightCategoryModel.id)
    .join(AgeCategoryModel, ApplicationModel.age_category_id ==
AgeCategoryModel.id)
    .join(RankModel, ApplicationModel.rank_id == RankModel.id)
    .join(ResultModel, ApplicationModel.id ==
ResultModel.participant_id)
    .where(ApplicationModel.competition_id == competition_id)
    .where(ApplicationModel.status == StatusCode.APPROVED)
    .order_by(
        AgeCategoryModel.id,
        WeightCategoryModel.id,
        (left_points_case + right_points_case).desc(),
        ApplicationModel.weight.asc()
    )
)

result = db.execute(query).all()

grouped_result = []
current_age_category = None
current_age_group = None
current_weight_category = None
current_weight_group = None
place_counter = 1

for row in result:
    if row.age_category != current_age_category:
        if current_weight_group:

```

```

current_age_group["weight_categories"].append(current_weight_group)
    if current_age_group:
        grouped_result.append(current_age_group)

    current_age_group = {
        "age_category": row.age_category,
        "weight_categories": []
    }
    current_age_category = row.age_category
    current_weight_category = None
    current_weight_group = None
    place_counter = 1

    if row.weight_category != current_weight_category:
        if current_weight_group:

current_age_group["weight_categories"].append(current_weight_group)

        current_weight_group = {
            "weight_category": row.weight_category,
            "participants": []
        }
        current_weight_category = row.weight_category
        place_counter = 1
    else:
        if current_weight_group["participants"]:
            prev = current_weight_group["participants"][-1]
            if not (prev["total_points"] == row.total_points and
                    prev["athlete_weight"] == row.athlete_weight):
                place_counter += 1
        else:
            pass

    full_name = f"{row.surname or ''} {row.name or ''}".strip()
    if row.patronymic:
        full_name += f" {row.patronymic}"

    current_weight_group["participants"].append({
        "place_by_two_arms": place_counter,
        "full_name": full_name,
        "gender": row.gender,
        "birth_date": row.birth_date,
        "rank": row.rank,
        "team": row.team,
        "left_hand_place": row.left_hand_place,
        "left_points": row.left_points,

```

```

        "right_hand_place": row.right_hand_place,
        "right_points": row.right_points,
        "total_points": row.total_points,
        "athlete_weight": row.athlete_weight
    })

    if current_weight_group:
        current_age_group["weight_categories"].append(current_weight_group)
    if current_age_group:
        grouped_result.append(current_age_group)

    return grouped_result

```

П.1.4.3 Код файла router.py

```

import os
from typing import Any

from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from starlette.responses import FileResponse

from src.auth.repository import UserRepository
from src.competitions.models import CompetitionModel
from src.results.service import WordExporter
from src.database.db import get_db
from src.results.repository import ResultsRepository

router = APIRouter(
    prefix="/results",
    tags=["results"],
)

@router.get("/{competition_id}")
def get_results(competition_id: int, db: Session = Depends(get_db)):
    results = ResultsRepository.get_results_for_competition(competition_id, db)
    return {
        "data": results
    }

@router.get("/{competition_id}/export")
def export_protocol_to_word(
    competition_id: int,
    db: Session = Depends(get_db)
):
    try:
        results = ResultsRepository.get_results_for_competition(
            competition_id=competition_id,

```

```

        db=db
    )
    if not results:
        raise HTTPException(status_code=404, detail="Результаты не найдены")

    competition = db.query(CompetitionModel).filter(
        CompetitionModel.id == competition_id
    ).first()
    if not competition:
        raise HTTPException(status_code=404, detail="Соревнование не
найдено")

    organizer = UserRepository.find_one_or_none_by_id(competition.organizer_id, db)
    organizer_name = f"{organizer.surname} {organizer.name}" if organizer
else 'не указан'
    if organizer and organizer.patronymic:
        organizer_name += f" {organizer.patronymic}"

    total_participants = 0
    male_count = 0
    female_count = 0
    for age_group in results:
        for weight_group in age_group["weight_categories"]:
            for participant in weight_group["participants"]:
                total_participants += 1
                gender = participant.get("gender")
                if gender == 1:
                    male_count += 1
                elif gender == 2:
                    female_count += 1

    output_path = WordExporter.export_competition_results_detailed(
        results=results,
        competition_name=competition.title,
        competition_date=competition.date.strftime('%d.%m.%Y') if
competition.date else 'не указана',
        competition_location=competition.venue,
        organizer=organizer_name,
        total_participants=total_participants,
        male_count=male_count,
        female_count=female_count
    )

    return FileResponse(
        path=output_path,
        filename=os.path.basename(output_path),

```



```

        media_type='application/vnd.openxmlformats-officedocument.wordprocessingml.document'
    )
except Exception as e:
    raise HTTPException(status_code=500, detail=f"Ошибка при экспорте: {str(e)}")

```

П.1.5 Код модуля турнирных сеток

П.1.5.1 Код файла service.py

```

from dataclasses import dataclass
from typing import Optional, List, Dict, Any
import asyncio
import json
from fastapi import WebSocket
from sqlalchemy.orm import Session

from src.tournament.repository import TournamentMatchRepository
from src.tournament.schemas import TournamentMatchSchema
from src.results.models import ResultModel
from src.tournament.models import TournamentModel

@dataclass
class TournamentParticipant:
    name: str
    application_id: int
    weight: Optional[float] = None
    wins: int = 0
    losses: int = 0
    eliminated: bool = False
    elimination_round: int = 0
    elimination_bracket: str = ""

    def add_loss(self, round_num: int, bracket: str):
        self.losses += 1
        if self.losses >= 2:
            self.eliminated = True
            self.elimination_round = round_num
            self.elimination_bracket = bracket

    def to_dict(self):
        return {
            "name": self.name,
            "wins": self.wins,
            "losses": self.losses,
            "eliminated": self.eliminated,
            "weight": self.weight,

```

```

        "application_id": self.application_id,
        "elimination_round": self.elimination_round,
        "elimination_bracket": self.elimination_bracket
    }

@dataclass
class Match:
    first_participant: Optional[TournamentParticipant]
    second_participant: Optional[TournamentParticipant]
    winner: Optional[TournamentParticipant] = None
    loser: Optional[TournamentParticipant] = None
    round_num: int = 0
    bracket: str = "winners"
    match_id: str = ""
    match_number: int = 0

    def is_bye(self) -> bool:
        return self.first_participant is None or self.second_participant is None

    def to_dict(self):
        return {
            "match_id": self.match_id,
            "first_participant": self.first_participant.name if
self.first_participant else None,
            "second_participant": self.second_participant.name if
self.second_participant else None,
            "winner": self.winner.name if self.winner else None,
            "loser": self.loser.name if self.loser else None,
            "round_num": self.round_num,
            "bracket": self.bracket,
            "is_bye": self.is_bye(),
            "match_number": self.match_number
        }

class Tournament:
    def __init__(self, participants: List[TournamentParticipant], db: Session,
competition_id: int,
                    tournament_id: str, hand: str, age_category_id: int,
weight_category_id: int):
        self.original_participants = participants.copy()
        self.current_round = 1
        self.waiting_for_choice = False
        self.current_match: Optional[Match] = None
        self.pending_choice: Optional[asyncio.Future] = None
        self.websocket: Optional[WebSocket] = None

```

```

self.db = db
self.competition_id = competition_id
self.tournament_id = tournament_id
self.hand = hand
self.age_category_id = age_category_id
self.weight_category_id = weight_category_id
self.match_counter = 0
self.is_finished = False
self._task: Optional[asyncio.Task] = None

def to_dict(self) -> Dict[str, Any]:
    return {
        "competition_id": self.competition_id,
        "tournament_id": self.tournament_id,
        "hand": self.hand,
        "age_category_id": self.age_category_id,
        "weight_category_id": self.weight_category_id,
        "current_round": self.current_round,
        "match_counter": self.match_counter,
        "is_finished": self.is_finished,
        "waiting_for_choice": self.waiting_for_choice,
        "original_participants": [p.to_dict() for p in
self.original_participants],
        "current_match": self.current_match.to_dict() if self.current_match
else None,
    }

@classmethod
async def from_dict(cls, data: Dict[str, Any], db: Session) -> "Tournament":
    participants = [
        TournamentParticipant(
            name=p["name"],
            application_id=p["application_id"],
            weight=p.get("weight"),
            wins=p["wins"],
            losses=p["losses"],
            eliminated=p["eliminated"],
            elimination_round=p.get("elimination_round", 0),
            elimination_bracket=p.get("elimination_bracket", "")
        )
        for p in data["original_participants"]
    ]
    tournament = cls(
        participants=participants,
        db=db,
        competition_id=data["competition_id"],
        tournament_id=data["tournament_id"],

```

```

        hand=data["hand"],
        age_category_id=data["age_category_id"],
        weight_category_id=data["weight_category_id"]
    )
    tournament.current_round = data["current_round"]
    tournament.match_counter = data["match_counter"]
    tournament.is_finished = data["is_finished"]
    tournament.waiting_for_choice = data["waiting_for_choice"]
    if data.get("current_match"):
        match_data = data["current_match"]
        first = next((p for p in participants if p.name ==
match_data["first_participant"]), None)
        second = next((p for p in participants if p.name ==
match_data["second_participant"]), None)
        tournament.current_match = Match(
            first_participant=first,
            second_participant=second,
            round_num=match_data["round_num"],
            bracket=match_data["bracket"],
            match_id=match_data["match_id"],
            match_number=match_data["match_number"]
        )
    return tournament

async def save_state_to_db(self, db: Session):
    state_dict = self.to_dict()
    active = db.query(TournamentModel).filter(
        TournamentModel.tournament_id == self.tournament_id
    ).first()
    if active:
        active.state_json = json.dumps(state_dict, ensure_ascii=False)
        active.is_active = True
        db.commit()

async def restore_state_from_db(self, db: Session):
    active = db.query(TournamentModel).filter(
        TournamentModel.tournament_id == self.tournament_id
    ).first()
    if active and active.state_json:
        state = json.loads(active.state_json)
        for i, p in enumerate(self.original_participants):
            saved = next((s for s in state["original_participants"] if
s["application_id"] == p.application_id), None)
            if saved:
                p.wins = saved["wins"]
                p.losses = saved["losses"]
                p.eliminated = saved["eliminated"]

```

```

        p.elimination_round = saved.get("elimination_round", 0)
        p.elimination_bracket = saved.get("elimination_bracket", "")
    self.current_round = state["current_round"]
    self.match_counter = state["match_counter"]
    self.is_finished = state["is_finished"]
    self.waiting_for_choice = state["waiting_for_choice"]
    if state.get("current_match"):
        match_data = state["current_match"]
        first = next((p for p in self.original_participants if p.name ==
match_data["first_participant"]), None)
        second = next((p for p in self.original_participants if p.name ==
match_data["second_participant"]), None)
        self.current_match = Match(
            first_participant=first,
            second_participant=second,
            round_num=match_data["round_num"],
            bracket=match_data["bracket"],
            match_id=match_data["match_id"],
            match_number=match_data["match_number"]
        )
    if self.waiting_for_choice and self.current_match:
        self.pending_choice = asyncio.Future()

    def _create_matches(self, participant_list:
List[Optional[TournamentParticipant]], bracket: str, round_num: int,
                        start_id: int) -> tuple[List[Match], int]:
        matches = []
        filtered = [p for p in participant_list if p and not p.eliminated]
        if len(filtered) % 2 != 0:
            filtered.append(None)
        for i in range(0, len(filtered), 2):
            self.match_counter += 1
            match = Match(
                first_participant=filtered[i],
                second_participant=filtered[i + 1] if i + 1 < len(filtered) else
None,
                round_num=round_num,
                bracket=bracket,
                match_id=f"{bracket}_{round_num}_{i//2}",
                match_number=self.match_counter
            )
            matches.append(match)
        return matches, start_id + len(matches)

    async def _ask_winner(self, match: Match) -> Optional[TournamentParticipant]:
        self.current_match = match
        self.waiting_for_choice = True

```

```

    if self.websocket:
        await self.websocket.send_json({
            "type": "match",
            "data": match.to_dict()
        })
    self.pending_choice = asyncio.Future()
    winner = await self.pending_choice
    self.waiting_for_choice = False
    self.current_match = None
    return winner

async def _save_match_result(self, match: Match):
    try:
        match_data = TournamentMatchSchema(
            competition_id=self.competition_id,
            tournament_id=self.tournament_id,
            hand=self.hand,
            age_category_id=self.age_category_id,
            weight_category_id=self.weight_category_id,
            round_num=match.round_num,
            bracket=match.bracket,
            match_number=match.match_number,
            first_participant_name=match.first_participant.name if
match.first_participant else "",
            second_participant_name=match.second_participant.name if
match.second_participant else None,
            winner_name=match.winner.name if match.winner else None,
            loser_name=match.loser.name if match.loser else None,
            is_bye=match.is_bye()
        )
        TournamentMatchRepository.add(self.db, **match_data.model_dump())
        await self._send_log(f"    [Сохранено] Матч {match.match_number}
({match.bracket})")
    except Exception as e:
        await self._send_log(f"    [Ошибка сохранения] {str(e)}")

async def _simulate_match(self, match: Match) -> None:
    if match.is_bye():
        match.winner = match.first_participant or match.second_participant
        match.loser = None
        await self._save_match_result(match)
        return
    winner = await self._ask_winner(match)
    if winner == match.first_participant:
        match.winner = match.first_participant
        match.loser = match.second_participant
    else:

```

```

        match.winner = match.second_participant
        match.loser = match.first_participant
        match.winner.wins += 1
        if match.loser:
            match.loser.add_loss(match.round_num, match.bracket)
        await self._save_match_result(match)

    async def _send_log(self, message: str):
        if self.websocket:
            try:
                await self.websocket.send_json({"type": "log", "data": message})
            except:
                pass

    async def _send_match_result(self, match: Match):
        if match.is_bye():
            await self._send_log(f"    BYE: {match.winner.name} автоматически
проходит дальше")
        else:
            await self._send_log(f"    {match.winner.name} победил
{match.loser.name}")

    async def _send_status_update(self):
        if self.websocket:
            status = {
                "type": "status",
                "data": {
                    "winners": [p.to_dict() for p in self.original_participants
if not p.eliminated and p.losses == 0],
                    "losers": [p.to_dict() for p in self.original_participants if
not p.eliminated and p.losses == 1],
                    "eliminated": [p.to_dict() for p in
self.original_participants if p.eliminated],
                    "current_round": self.current_round
                }
            }
            await self.websocket.send_json(status)

    async def _run_round_matches(self, matches: List[Match], bracket_name: str)
-> tuple[List[TournamentParticipant], List[TournamentParticipant]]:
        winners = []
        losers = []
        await self._send_log(f"\n{'='*60}")
        await self._send_log(f"ПАУНД {self.current_round} - {bracket_name}")
        await self._send_log(f"\n{'='*60}")
        for i, match in enumerate(matches, 1):
            await self._send_log(f"\n--- Матч {i} из {len(matches)} ---")

```

```

        await self._simulate_match(match)
        await self._send_match_result(match)
        if match.winner:
            winners.append(match.winner)
        if match.loser and not match.loser.eliminated:
            losers.append(match.loser)
        await self._send_status_update()
        await self.save_state_to_db(self.db)
    return winners, losers

async def run_tournament(self, websocket: WebSocket):
    self.websocket = websocket
    await self._send_log("\n" + "="*60)
    await self._send_log("ТУРНИР С ДВОЙНЫМ ВЫБЫВАНИЕМ (Double Elimination)")
    await self._send_log("="*60)
    await self._send_log("ПРАВИЛА: Участник выбывает после двух поражений")

    winners_pool = [p for p in self.original_participants if not p.eliminated]
    losers_pool = []
    round_num = 1

    while True:
        self.current_round = round_num
        winners_matches_played = False
        losers_matches_played = False

        if winners_pool:
            winners_matches, _ = self._create_matches(winners_pool,
"winners", round_num, 0)
            if winners_matches:
                new_winners, losers_from_winners = await
self._run_round_matches(winners_matches, "СЕТКА ПОБЕДИТЕЛЕЙ")
                winners_pool = new_winners
                losers_pool.extend(losers_from_winners)
                winners_matches_played = True

        if losers_pool:
            losers_matches, _ = self._create_matches(losers_pool, "losers",
round_num, 0)
            if losers_matches:
                new_losers, eliminated = await
self._run_round_matches(losers_matches, "СЕТКА ПРОИГРАВШИХ")
                losers_pool = new_losers
                losers_matches_played = True

        if winners_matches_played or losers_matches_played:

```



```

        await self._send_log(f"\n--- СТАТУС ПОСЛЕ РАУНДА {round_num} ---")
    ")

    if winners_pool:
        await self._send_log(f"  Winners: {' '.join(p.name for p in
winners_pool)}")
    else:
        await self._send_log("  Winners: пусто")
    if losers_pool:
        await self._send_log(f"  Losers: {' '.join(p.name for p in
losers_pool)}")
    else:
        await self._send_log("  Losers: пусто")

    if len(winners_pool) == 1 and len(losers_pool) == 1:
        await self._send_log("ГРАНД-ФИНАЛ!")
        await self._send_log(f"Чемпион winners: {winners_pool[0].name} (0
поражений)")
        await self._send_log(f"Чемпион losers: {losers_pool[0].name} (1
поражение)")

    self.match_counter += 1
    final_match = Match(
        first_participant=winners_pool[0],
        second_participant=losers_pool[0],
        round_num=round_num+1,
        bracket="grand_final",
        match_id="grand_final_1",
        match_number=self.match_counter
    )
    await self._send_log(f"\n--- ГРАНД-ФИНАЛ, МАТЧ 1 ---")
    await self._simulate_match(final_match)
    await self._send_match_result(final_match)
    champion = final_match.winner

    if final_match.winner == losers_pool[0]:
        await self._send_log("Победитель из losers выиграл первый
финал! Нужен решающий матч.")
        self.match_counter += 1
        final_match2 = Match(
            first_participant=winners_pool[0],
            second_participant=losers_pool[0],
            round_num=round_num+1,
            bracket="grand_final",
            match_id="grand_final_2",
            match_number=self.match_counter
        )
        await self._send_log(f"\n--- ГРАНД-ФИНАЛ, МАТЧ 2 ---")

```

```

        await self._simulate_match(final_match2)
        await self._send_match_result(final_match2)
        champion = final_match2.winner

        await self._send_log(f"ЧЕМПИОН ТУРНИРА: {champion.name}")
        await self._show_final_stats()
        self.is_finished = True
        await self.save_state_to_db(self.db)
        if self.websocket:
            await self.websocket.send_json({
                "type": "tournament_finished",
                "data": {"champion": champion.name, "hand": self.hand}
            })
        return {"champion": champion.name, "stats": [p.to_dict() for p in
self.original_participants]}

    if len(winners_pool) == 1 and not losers_pool:
        champion = winners_pool[0]
        await self._send_log(f"ЧЕМПИОН ТУРНИРА: {champion.name}
(НЕПОВЕЖДЁННЫЙ!) ")
        await self._show_final_stats()
        self.is_finished = True
        await self.save_state_to_db(self.db)
        if self.websocket:
            await self.websocket.send_json({
                "type": "tournament_finished",
                "data": {"champion": champion.name, "hand": self.hand}
            })
        return {"champion": champion.name, "stats": [p.to_dict() for p in
self.original_participants]}

    if not winners_pool and not losers_pool:
        await self._send_log("\nТурнир завершён без чемпиона?")
        self.is_finished = True
        await self.save_state_to_db(self.db)
        break

    round_num += 1

    return {"champion": None, "stats": [p.to_dict() for p in
self.original_participants]}

async def _show_final_stats(self):
    await self._send_log("\n" + "="*60)
    await self._send_log("ИТОГОВАЯ СТАТИСТИКА ВСЕХ УЧАСТНИКОВ")
    await self._send_log("="*60)
    for p in self.original_participants:

```

```

        status = "ВЫБЫЛ" if p.eliminated else "АКТИВЕН"
        await self._send_log(f"{p.name}: {p.wins} побед, {p.losses} поражений
- {status}")

    async def make_choice(self, winner_name: str):
        if self.pending_choice and not self.pending_choice.done():
            winner = next((p for p in self.original_participants if p.name ==
winner_name), None)
            if winner:
                self.pending_choice.set_result(winner)

    def calculate_placements(self) -> Dict[int, int]:
        def sort_key(p: TournamentParticipant):
            return (
                p.losses,
                -p.elimination_round if p.losses == 1 else 0,
                p.weight if p.weight is not None else 999,
                -p.wins
            )
        sorted_participants = sorted(self.original_participants, key=sort_key)
        placements = {}
        for i, p in enumerate(sorted_participants, 1):
            placements[p.application_id] = i
        return placements

    async def save_results_to_db(self, db: Session) -> dict:
        placements = self.calculate_placements()
        results = {}
        for app_id, place in placements.items():
            existing = db.query(ResultModel).filter(ResultModel.participant_id
== app_id).first()
            if self.hand == "left":
                if existing:
                    existing.left_hand_place = place
                else:
                    new = ResultModel(participant_id=app_id,
left_hand_place=place, right_hand_place=None)
                    db.add(new)
            else:
                if existing:
                    existing.right_hand_place = place
                else:
                    new = ResultModel(participant_id=app_id,
left_hand_place=None, right_hand_place=place)
                    db.add(new)
            results[app_id] = place
        try:

```

```

        db.commit()
    except Exception as e:
        db.rollback()
        raise e
    return results

```

П.1.5.2 Код файла router.py

```

import json

from fastapi import APIRouter, WebSocket, WebSocketDisconnect, Depends,
HTTPException, Query
from sqlalchemy.orm import Session
import asyncio

from src.database.db import get_db
from src.applications.models import ApplicationModel
from src.applications.types import StatusCode
from src.auth.models import UserModel
from src.auth.router import check_user_role
from src.auth.schemas import Role
from src.auth.jwt_auth.base.auth import JWTAuth
from src.auth.jwt_auth.base.config import JWTConfig
from src.auth.jwt_auth.utils import try_to_decode_token
from src.tournament.service import Tournament, TournamentParticipant
from src.tournament.schemas import StartTournamentRequest,
StartTournamentResponse, TournamentMatchSchema
from src.tournament.repository import TournamentMatchRepository,
ActiveTournamentRepository
from src.tournament.models import TournamentModel, MatchModel

router = APIRouter(prefix="/tournament", tags=["tournament"])
active_tournaments = {}
jwt_auth = JWTAuth(config=JWTConfig())

@router.post("/start/{competition_id}", response_model=StartTournamentResponse)
async def start_tournament(
    competition_id: int,
    req: StartTournamentRequest,
    db: Session = Depends(get_db),
    current_user = Depends(check_user_role(Role.ORGANIZER))
):
    for tid, t in active_tournaments.items():
        if (t.competition_id == competition_id and
            t.age_category_id == req.age_category_id and
            t.weight_category_id == req.weight_category_id and
            t.hand == req.hand):

```

```

        raise HTTPException(status_code=409, detail=f"Турнир уже существует
(ID: {tid}))")
    existing = db.query(TournamentModel).filter(
        TournamentModel.competition_id == competition_id,
        TournamentModel.age_category_id == req.age_category_id,
        TournamentModel.weight_category_id == req.weight_category_id,
        TournamentModel.hand == req.hand,
        TournamentModel.is_active == True
    ).first()
    if existing:
        raise HTTPException(status_code=409, detail=f"Турнир уже существует (ID:
{existing.tournament_id})")

    applications = db.query(ApplicationModel).filter(
        ApplicationModel.competition_id == competition_id,
        ApplicationModel.status == StatusCode.APPROVED,
        ApplicationModel.age_category_id == req.age_category_id,
        ApplicationModel.weight_category_id == req.weight_category_id
    ).all()
    if len(applications) < 2:
        raise HTTPException(status_code=400, detail="Недостаточно участников")
    participants = []
    for app in applications:
        user = db.query(UserModel).filter(UserModel.id == app.user_id).first()
        if user:
            full_name = f"{user.surname} {user.name} {user.patronymic}".strip()
            weight = float(app.weight) if app.weight else None
            participants.append(TournamentParticipant(name=full_name,
application_id=app.id, weight=weight))
        else:
            full_name = f"{app.surname} {app.name} {app.patronymic}".strip()
            weight = float(app.weight) if app.weight else None
            participants.append(TournamentParticipant(name=full_name,
application_id=app.id, weight=weight))
    if len(participants) < 2:
        raise HTTPException(status_code=400, detail="Не удалось собрать
участников")

    tournament_id =
f"tournament_{competition_id}_{req.age_category_id}_{req.weight_category_id}_{req.hand
}_{len(active_tournaments)+1}"
    tournament = Tournament(participants, db, competition_id, tournament_id,
req.hand, req.age_category_id, req.weight_category_id)
    active_tournaments[tournament_id] = tournament

    active_record = TournamentModel(
        tournament_id=tournament_id,

```

```

        competition_id=competition_id,
        organizer_id=current_user.id,
        hand=req.hand,
        age_category_id=req.age_category_id,
        weight_category_id=req.weight_category_id,
        participants_data=json.dumps([{"name": p.name, "app_id":
p.application_id, "weight": p.weight} for p in participants]),
        is_active=True
    )
    db.add(active_record)
    db.commit()
    await tournament.save_state_to_db(db)
    return StartTournamentResponse(tournament_id=tournament_id)

@router.websocket("/ws/{tournament_id}")
async def websocket_endpoint(
    websocket: WebSocket,
    tournament_id: str,
    token: str = Query(...),
    db: Session = Depends(get_db)
):
    try:
        payload = try_to_decode_token(jwt_auth, token)
        role_id = payload.get("role_id")
        if role_id < 2:
            await websocket.close(code=1008, reason="Forbidden")
            return
    except Exception:
        await websocket.close(code=1008, reason="Invalid token")
        return
    await websocket.accept()

    tournament = active_tournaments.get(tournament_id)
    if tournament is None:
        active_record = db.query(TournamentModel).filter(
            TournamentModel.tournament_id == tournament_id,
            TournamentModel.is_active == True
        ).first()
        if not active_record:
            await websocket.send_json({"type": "error", "data": "Tournament not
found"})

            await websocket.close(code=1008)
            return
        participants_data = json.loads(active_record.participants_data)
        participants = [

```

```

        TournamentParticipant(name=p["name"],      application_id=p["app_id"],
weight=p.get("weight"))
        for p in participants_data
    ]
    tournament = Tournament(
        participants=participants,
        db=db,
        competition_id=active_record.competition_id,
        tournament_id=tournament_id,
        hand=active_record.hand,
        age_category_id=active_record.age_category_id,
        weight_category_id=active_record.weight_category_id
    )
    await tournament.restore_state_from_db(db)
    active_tournaments[tournament_id] = tournament

tournament.websocket = websocket
if tournament.waiting_for_choice and tournament.current_match:
    tournament.pending_choice = asyncio.Future()
    await websocket.send_json({
        "type": "match",
        "data": tournament.current_match.to_dict()
    })

try:
    if not hasattr(tournament, "_task") or tournament._task is None or
tournament._task.done():
        tournament._task =
        asyncio.create_task(tournament.run_tournament(websocket))

    while True:
        data = await websocket.receive_text()
        message = json.loads(data)
        if message["type"] == "choice":
            await tournament.make_choice(message["winner"])
            await tournament.save_state_to_db(db)
except WebSocketDisconnect:
    await tournament.save_state_to_db(db)
    tournament.websocket = None
    if hasattr(tournament, "_task") and not tournament._task.done():
        tournament._task.cancel()
except Exception as e:
    print(f"WebSocket error: {e}")
    await tournament.save_state_to_db(db)
finally:
    pass

```

```

@router.get("/matches/{competition_id}")
def get_tournament_matches(
    competition_id: int,
    db: Session = Depends(get_db),
    current_user = Depends(check_user_role(Role.ORGANIZER))
):
    matches = TournamentMatchRepository.find_all(db,
competition_id=competition_id)
    return {"data": [TournamentMatchSchema.model_validate(m).model_dump() for m
in matches]}

```

```

@router.post("/save/{tournament_id}")
async def save_tournament_results(
    tournament_id: str,
    db: Session = Depends(get_db),
    current_user = Depends(check_user_role(Role.ORGANIZER))
):
    if tournament_id not in active_tournaments:
        raise HTTPException(status_code=404, detail="Турнир не найден")
    tournament = active_tournaments[tournament_id]
    if not tournament.is_finished:
        raise HTTPException(status_code=400, detail="Турнир не завершён")
    try:
        results = await tournament.save_results_to_db(db)
        if tournament_id in active_tournaments:
            del active_tournaments[tournament_id]
        active_record =
db.query(TournamentModel).filter(TournamentModel.tournament_id
tournament_id).first()
        if active_record:
            active_record.is_active = False
            db.commit()
        return {"message": "Результаты сохранены", "results": results}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Ошибка сохранения:
{str(e)}")

```

```

@router.get("/check/{competition_id}")
def check_active_tournament(
    competition_id: int,
    age_category_id: int,
    weight_category_id: int,
    hand: str,
    db: Session = Depends(get_db),

```



```

):
    active = db.query(TournamentModel).filter(
        TournamentModel.competition_id == competition_id,
        TournamentModel.age_category_id == age_category_id,
        TournamentModel.weight_category_id == weight_category_id,
        TournamentModel.hand == hand,
        TournamentModel.is_active == True
    ).first()
    if active:
        return {"exists": True, "tournament_id": active.tournament_id}
    return {"exists": False}

@router.get("/bracket/{competition_id}")
def get_tournament_bracket(
    competition_id: int,
    age_category_id: int,
    weight_category_id: int,
    hand: str,
    db: Session = Depends(get_db),
):
    matches = db.query(MatchModel).filter(
        MatchModel.competition_id == competition_id,
        MatchModel.age_category_id == age_category_id,
        MatchModel.weight_category_id == weight_category_id,
        MatchModel.hand == hand
    ).all()

    winners = [m for m in matches if m.bracket == "winners"]
    losers = [m for m in matches if m.bracket == "losers"]
    grand = [m for m in matches if m.bracket == "grand_final"]

    winners_by_round = {}
    for m in winners:
        winners_by_round.setdefault(m.round_num, []).append(m)

    losers_by_round = {}
    for m in losers:
        losers_by_round.setdefault(m.round_num, []).append(m)

    all_rounds = sorted(set(winners_by_round.keys()) |
set(losers_by_round.keys()))
    rounds_data = []
    for r in all_rounds:
        rounds_data.append({
            "round_num": r,
            "winners_matches": [

```

```

        {
            "id": m.id,
            "first": m.first_participant_name,
            "second": m.second_participant_name,
            "winner": m.winner_name,
            "is_bye": m.is_bye
        } for m in winners_by_round.get(r, [])
    ],
    "losers_matches": [
        {
            "id": m.id,
            "first": m.first_participant_name,
            "second": m.second_participant_name,
            "winner": m.winner_name,
            "is_bye": m.is_bye
        } for m in losers_by_round.get(r, [])
    ]
}

grand_final_matches = []
for m in grand:
    grand_final_matches.append({
        "match_number": m.match_number,
        "first": m.first_participant_name,
        "second": m.second_participant_name,
        "winner": m.winner_name,
        "is_bye": m.is_bye
    })

return {
    "data": {
        "rounds": rounds_data,
        "grand_final_matches": grand_final_matches
    }
}

```

ПРИЛОЖЕНИЕ 2

РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

П.2.1. Назначение системы

Назначением системы для организации соревнований по армрестлингу является обеспечение автоматизации регистрации участников, ведения турнирных сеток, а также создания итоговых протоколов по результатам турниров.

П.2.2. Условия применения системы

Клиентская часть приложения доступна для компьютеров с установленным веб-браузером. Для работы серверной части приложения необходим сервер на ОС GNU/Linux с установленным ПО Docker и Docker Compose. Минимально допустимый для работы системы сервер должен иметь процессор с 4 физическими ядрами на частоте не менее 2.8 ГГц, 8 Гб оперативной памяти и жёстким диском объёмом не менее 100 Гб.

П.2.3. Подготовка системы к работе

Для начала работы с системой пользователь должен зарегистрироваться в системе. По умолчанию новый зарегистрированный пользователь получает роль участника. Для получения роли организатора нужно лицо с доступом к базе данных. Неавторизованный пользователь тоже имеет доступ к системе с ограниченным функционалом.

П.2.4. Описание операций

П.2.4.1. Просмотр списка соревнований

Данная операция доступна для всех пользователей. При переходе на основную страницу веб-приложения на клиентской части, пользователь автоматически перенаправляется на страницу со списком соревнований.

Страница с соревнованиями

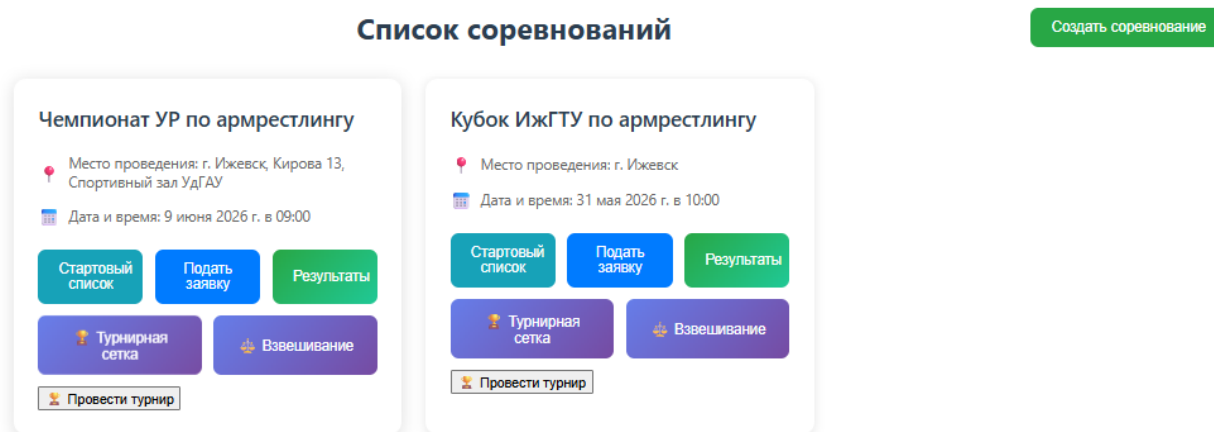


Рис. П.2.1

П.2.4.2. Просмотр стартового списка участников соревнования

Данная операция доступна для всех пользователей. На карточке соревнования необходимо нажать на кнопку «Стартовый список». После нажатия на кнопку пользователь перейдет на страницу со списком участников, которые подали заявки на участие в соревнованиях.

Стартовый список

[← Назад](#)
Стартовый список: Чемпионат УР по армрестлингу

Всего участников: 8

Юниоры (19-21 лет)				
85 кг				
№	ФАМИЛИЯ	ИМЯ	РАЗРЯД	КОМАНДА
1	Егоров	Иван	2 раз.	ИжГТУ
2	Егоров	Петр	без разряда	ИжГТУ
3	Иванов	Максим	1 раз.	ИжГТУ
4	Иванов	Николай	МС	ИжГТУ
5	Максимов	Егор	3 раз.	ИжГТУ
6	Максимов	Егор	2 юн. раз.	ИжГТУ
7	Петров	Иван	3 юн. раз.	ИжГТУ
8	Петров	Максим	1 юн. раз	ИжГТУ

Рис. П.2.2

П.2.4.3. Подача заявки на участие в соревновании

Данный процесс доступен только для авторизованных пользователей с ролью «Участник». На карточке соревнования необходимо нажать на кнопку «Подать заявку». Далее пользователя перенаправляет на страницу с формой для заявки. После ввода всех данных, необходимо нажать кнопку «Подать заявку». После этого заявка будет успешно отправлена.

Подача заявки на соревнование

Подача заявки на соревнование

Возрастная категория:

Юниоры (19-21 лет) (2005 г.р. - 2007 г.р.) ▼

Весовая категория:

80 кг (75 - 80 кг) ▼

Спортивное звание:

КМС ▼

Команда:

ИжГТУ

Копия паспорта:

Выберите файл копия паспорта.pdf

Файл страхового полиса:

Выберите файл страховой полис.pdf

☒ Я даю согласие на обработку моих персональных данных

Подать заявку

Рис. П.2.3

П.2.4.4. Просмотр и обработка заявок участников

Данный процесс доступен авторизованным пользователям с ролью «Организатор». На шапке веб-приложения необходимо нажать кнопку «Все заявки». После этого действия организатора перенаправляет на страницу с заявками всех участников. Здесь он проверяет корректность данных участников и в зависимости от этого меняет статус заявки. Также у организатора есть возможность отправить сообщение на электронную почту участника с информацией о заявке.

Просмотр заявок

Все заявки участников

Соревнование: Чемпионат УР по армрестлингу
ФИО участника: Иванов Максим Петрович
Возрастная категория: Юниоры (19-21 лет)
Весовая категория: 85 кг
Команда: ИжГТУ
Создана: 05:49, 08 мая 2026
Обновлена: 05:49, 08 мая 2026
Статус: Неизвестно

Нет прикрепленных файлов.

Изменить статус: Принята ▼

Соревнование: Чемпионат УР по армрестлингу
ФИО участника: Егоров Петр Егорович
Возрастная категория: Юниоры (19-21 лет)
Весовая категория: 85 кг
Команда: ИжГТУ
Создана: 05:49, 08 мая 2026
Обновлена: 05:49, 08 мая 2026
Статус: Неизвестно

Нет прикрепленных файлов.

Изменить статус: Принята ▼

Рис. П.2.4

П.2.4.5. Создание и ведение турнирной сетки

Данная операция доступна для авторизованных пользователей с ролью «Организатор». На карточке соревнования необходимо нажать на кнопку «Провести турнир». Далее пользователя перенаправляет на страницу с выбором категорий для начала турнирной сетки. После выбора категорий необходимо нажать кнопку «Начать турнирную сетку». Организатор перенаправляется на страницу ведения турнирной сетки, где он может выбирать победителей поединков. После окончания турнирной сетки, организатору необходимо нажать на кнопку «Сохранить результаты», чтобы данные о местах, занятых участниками, были записаны в базу данных.

Ведение турнирной сетки

← Назад к соревнованию

🏆

Турнирная сетка (Левая рука)

Раунд: 2

✔ Сетка победителей (0 поражений)

Иванов Максим Петрович(2-0)

Максимов Егор Петрович(2-0)

⚠ Сетка проигравших (1 поражение)

Егоров Петр Егорович(2-1)

Иванов Николай Иванович(2-1)

💀 Выбывшие (2 поражения)

Максимов Егор Иванович(0-2)

Петров Иван Максимович(1-2)

Петров Максим Иванович(0-2)

Егоров Иван Егорович(1-2)

🔥 Текущий матч

Сетка победителей Раунд 3

Иванов Максим Петрович

VS

Максимов Егор Петрович

Нажмите на имя победителя

Рис. П.2.5

П.2.4.6. Просмотр итогов соревнований

Данная операция доступна для всех пользователей. На карточке соревнования нужно нажать на кнопку «Результаты». После этого идет перенаправление на страницу с таблицей результатов участников для каждой категории. Для удобства итоги соревнований можно экспортировать в виде Word-файла.

Просмотр итогов соревнований

← Назад

Итоговые результаты: Чемпионат УР по армреслингу

Всего участников: 8

Скачать протокол (Word)

Юниоры (19-21 лет)

85 кг 8 участн.

МЕСТО ПО "ДВОЕБОРЬЮ"	ФАМИЛИЯ, ИМЯ, ОТЧЕСТВО	ПОЛ, ДАТА РОЖДЕНИЯ	СПОРТИВНОЕ ЗВАНИЕ	КОМАНДА	МЕСТО В БОРЬБЕ ЛЕВОЙ РУКОЙ	ОЧКИ	МЕСТО В БОРЬБЕ ПРАВОЙ РУКОЙ	ОЧКИ	СУММА ОЧКОВ	ВЕС СПОРТСМЕНА
1	Максимов Егор Петрович	муж., 04.07.2004	2 юн. раз.	ИЖГТУ	1	25		0	25	84.23
2	Петров Иван Максимович	муж., 04.07.2004	3 юн. раз.	ИЖГТУ	2	17		0	17	81.39
3	Максимов Егор Иванович	муж., 04.07.2004	3 раз.	ИЖГТУ	3	9		0	9	81.41
4	Иванов Максим Петрович	муж., 04.07.2004	1 раз.	ИЖГТУ	4	5		0	5	81.55
5	Егоров Иван Егорович	муж., 04.07.2004	2 раз.	ИЖГТУ	5	3		0	3	81.74
6	Петров Максим Иванович	муж., 04.07.2004	1 юн. раз.	ИЖГТУ	6	2		0	2	81.95
7	Егоров Петр Егорович	муж., 04.07.2004	без разряда	ИЖГТУ	7	0		0	0	82.16
8	Иванов Николай Иванович	муж., 04.07.2004	МС	ИЖГТУ	8	0		0	0	83.17

Рис. П.2.6